claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a671e176211cacea5.jsonl`
- SHA-256: `bf9111510ac74c51e532d0bd68554bb9a77c30cae85545de5086beed022dbfeb`
- Source modified: `2026-06-14T08:55:59+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T08:52:35.864Z)

Produce a DETAILED, PR-SIZED implementation plan for the PERSONALIZATION FEATURE in the badminton-highlight-indexer repo (cwd). The owner adopted the HYBRID: a generalization-first baseline + a flag-gated per-user personalization FEATURE on top, with a CONTRIBUTION FLYWHEEL. Ground the plan in the real code/docs ? read: backend/eval/experiment.py (Variant/compare/LOVO/leaderboard), backend/eval/eval_stats.py (honest small-N stats), backend/eval/distill_local.py (per-corpus model fit + LogisticRegression save/load), backend/pipeline/segmenters/fusion_hybrid.py + backend/config/models.py (FusionConfig cue_flags + Gate thresholds + how served config is read), backend/infrastructure/database.py (schema, the `human_label` column, jobs), backend/main.py (the API surface), and scratch/PERSONALIZATION_PIVOT_ANALYSIS.md (the strategy + the gap-list this plan must FUND).

FOUNDATION PRINCIPLE (owner): "a tool to help people which GETS BETTER THE MORE YOU HELP IT." Contribution (data / video / diagnostics / annotation) drives BOTH the user's own personalization AND the shared baseline; payoff SCALES WITH CONTRIBUTION.

Design these constructs (gap-list + owner additions):
1. PER-USER STORE + SERVING BRIDGE ? a user/tenant dimension + per-user model store (LogReg coeffs + thresholds + cue_flags + version); wire `LogisticRegression.load` into the served FusionSegmenter decision path (it is NEVER called in production today ? the #1 wall). Per-user model versioning that survives baseline upgrades.
2. CORRECTION-LOOP ? PER-USER LABELS ? the write path: a confirm/reject API that SETS `human_label` (currently a read-only stub with no setter), turning corrections into per-user labels; a recall-protecting de-bias prior (corrections are biased: users flag false-positives >> silent misses).
3. PER-USER PROMOTION GATE ? integrate the safety construct being built in branch feat/per-user-promotion-gate (min-N floor + structural-guard exemption); surface per-user cue ON/OFF as a TRUST feature ("we tested motion on YOUR videos ? it didn't help, so it's off").
4. CONTRIBUTION FLYWHEEL ? opt-in upload of data/videos/DIAGNOSTICS to improve the SHARED baseline; design payoff-scales-with-contribution (reproduce capture settings + more videos => more personalization benefit; single-video user leans on baseline). Consent ledger + minors-exclusion + opt-in governance. The baseline is owner-trained on a powerful GPU.
5. USER-AS-ANNOTATOR CHARTER ? integrate the sibling rally-annotator: internally GENERATE a short ~2-min segment from one of the user's OWN videos (active-learning: pick high-uncertainty regions) and guide the user to rally-label it occasionally to improve grounding; design cadence/UX/trigger + how labels feed per-user tuning AND optionally the baseline.
6. PER-USER EVAL ? the "<5 videos to superior quality" as a FALSIFIABLE held-out-USER benchmark (the release criterion); a per-user no-regression guard (none exists today).
7. DAY-0 BATTERIES = the validated label-free Gate-A/B fusion (zero per-user labels: ?F1 +0.074, 6/6, p=0.031); baseline-upgrade governance (seed/Roora + opt-in pooling + optional federated, honoring the analysis's privacy/consent constraints).

OUTPUT: write the full design to scratch/PERSONALIZATION_FEATURE_DESIGN.md (architecture, per-user data model + schema migrations, serving bridge, APIs, each construct, governance) AND a PHASED PR-SIZED BUILD PLAN: Phase 0 = safety + day-0 batteries (buildable THIS WEEKEND to rehearse); Phase 1 = per-user tuning serving + store; Phase 2 = correction-loop labels +

contribution flywheel; Phase 3 = annotator charter + shot taxonomy + baseline upgrades. Each phase lists concrete small mergeable PRs (files touched + tests) and marks engineering vs owner-gated. Honor the analysis's risks honestly (small-N eval noise, cold-start, privacy-deferral, compute).

Return: a ~300-word executive summary + the Phase-0 PR list (what to build this weekend) + the top 5 open design questions for the owner. Do NOT write code or open PRs ? this is a PLAN only.

2. assistant (2026-06-14T08:52:38.673Z)

I'll explore the codebase thoroughly before designing the plan. Let me start by reading the key files mentioned in the requirements.

3. user (2026-06-14T08:52:39.643Z)

1 # Personalization-First Pivot ? Decision-Grade Strategy Proposal
2
3 **Status:** Owner-gated decision doc. Strategy/product ? NOT an implementation authorization.
4 **Date:** 2026-06-14 · **Author:** Claude (analysis), for owner decision
5 **Discipline:** Honest measured results only. Every load-bearing empirical claim below is
6 verified against the live repo this session (see *Evidence Appendix*). Where a claim is an
7 assertion or unmeasured, it is labeled as such.
8
9 ---
10
11 ## 1. The pivot thesis, crystallized
12
13 Instead of shipping **one universal model** for all users (generalization-first), ship a
14 **batteries-included baseline that continually self-optimizes to a single user's own footage**.
15 As a user's corpus grows (10?15 videos), the on-device decision layer + **per-user cue selection**
16 ? driven by the existing **LOVO / A-B experiment harness run on the user's OWN videos**, with the
17 **correction loop as the per-user label source** ? becomes heavily tuned to *that* user's rallies and
18 shot types (serves, smashes, drives, drops). Per-user decisions like *"the motion/person cue gives no
19 F1 win for THIS user ? default it OFF for them."* Periodic **"batteries upgrades"** improve the shipped
20 baseline so a new user needs only **<5 videos** to reach superior quality. Two archetypes: **(a)** a
21 generous hobbyist who records some days; **(b)** a professional with 100+ hours of their own video. The
22 owner believes the harness + eval + A/B were *designed* to enable this. **This proposal finds that belief
23 is true at the mechanism layer for per-user TUNING, materially overstated for per-user LEARNING and as a
24 business strategy, and that the headline "<5 videos" promise is ? provably ? a *generalization* metric.**
25
26 ---
27
28 ## 2. PROS / CONS table
29
30 | Dimension | PROS (for personalization-first) | CONS (against) |
31 |---|---|---|
32 | **Product / PMF** | Sharpest ownable value prop in the field ? *"your videos, your model, on your machine."* Every incumbent (SwingVision $179.99/yr, BadPro+, Dartfish, Hudl) is cloud + generic. Privacy/ownership is a real edge-AI tailwind. | **Cold-start kills the median user**: value materializes at 10?15 videos; incumbents deliver on clip #1. Re-centers on the **individual the validated PMF research deprioritized** (academies #1 *because* amateurs are

low-WTP/high-churn). "Self-optimizes to me" is a faith claim a hobbyist **can't evaluate at purchase**. |
33 | **Market / moat** | Per-user tuned model + correction history = **sticky, non-portable switching cost**. 100+hr professional is a strong, underserved fit (corpus size ? WTP). | **Trades a compounding cross-user data flywheel** (the strategy docs' *stated* moat) **for a non-compounding per-seat tool**. User #1,000 gets nothing from users #1?999. Tool business, not platform business. |
34 | **Technical (decision layer)** | The **"detect once, decide forever"** split is real and shipped: detection persists to `candidate_segments.stats`; any decision variant re-scores offline, no GPU. Per-user personalization == per-user re-scoring. Personalizable artifacts are **tiny** (?5-feature numpy LogReg, kilobyte JSON; ~1.1M-param Gen-0 head trains in minutes on a 2070 Super). | **Per-user A/B is the n=6 problem, WORSE.** `eval_stats.py` exists *because* a bare LOVO mean lies at n?6; per-user runs at n=1?15 where LOVO is undefined?noisy. The first real A/B already produced **F1=0.000** (predicted zero rallies ? operating-point artifact at small N). |
35 | **Technical (heavy layer)** | Frozen WASB backbone ? catastrophic forgetting is **designed around** for the cheap per-user head. Cold-start has a real fallback: **label-free Gate-A/Gate-B** (zero per-user labels needed). | **Qwen3-VL fine-tune is infeasible on a hobbyist box** (needs rented 24?48 GB; 2070 Super "cannot QLoRA a 7B video VLM"; even 4B *inference* latency is unmeasured). VLM personalization ? cloud bursts ? **re-opens DPA/consent/no-train-on-minors** the local pivot meant to escape. |
36 | **Data / labels / privacy** | A user's own footage is **commercial-clean by construction** ? sidesteps the paid-LLM-not-a-training-source rule and provenance debt. On-device = the strongest privacy story available; GDPR/DPDP largely N/A for the per-user loop. | **Privacy is true ONLY on-device, which is explicitly DEFERRED.** Near-term (hosted-API premise) = per-user video + labels **in the cloud being trained on** ? the exact surface the pivot advertises escaping. The privacy thesis **writes a check only the deferred milestone can cash.** Correction-loop labels are **biased** (users flag false-positives ? silent misses). |
37 | **Shot taxonomy** | Clean *third* layer on the same persisted trajectory + person tracks; coarse hard/soft cut derivable from **existing** speed/arc/direction-reversal signals, no new model. Strong fit for archetype (b). | **Entirely greenfield** ? zero shot-type detection code today. **Class imbalance** (minority smash/drop/net may be near-absent in 15 hobbyist videos), not just small N. **Correction loop yields no shot labels** (rally-level only) ? needs a new, higher-effort per-shot UI the hobbyist won't absorb. |
38 | **"Harness was designed for this"** | **True for per-user TUNING**: `compare(videos, control, variant)` is corpus-relative; cue flags / Gate thresholds / honest small-N stats / replayable feature JSONs all compose into per-user cue selection with little new code. | **Overstated for LEARNING + as strategy**: learned per-user model has **no serving path**; correction?label **write path is a stub**; **no user/tenant dimension** (single-corpus by design). No doc/test/comment names personalization. "Bones support it" ?; "designed for it" = retrospective fit. |
39
40   ---
41
42   ## 3. How the existing harness + owned-model roadmap support it (concrete)
43
44   ### Genuinely real and reusable AS-IS (the cheap, near-ready 20%)
45   - **Corpus-relative LOVO A/B.** `backend/eval/experiment.py::compare()` / `calibration.leave_one_video_out` take an arbitrary video list ? "this user's corpus" is literally that list. `compare(user_corpus, control, person_off)` answers *"does the person cue win for THIS user?"* with **zero new code**. The pivot's flagship decision (*"motion cue OFF for this user"*) is exactly this.
46   - **Per-cue gating is a first-class knob.** `Variant.cue_flags`, `min_crossings` (Gate A), `min_players` (Gate B) are the per-user switches; the served `FusionSegmenter` already reads `indexing.fusion.min_crossings/min_players` from per-deployment config. **Single-user-per-deployment (the hobbyist) gets per-user cue selection essentially for free.**
47   - **Honest small-N gate.** `eval_stats.py` (bootstrap ?F1 CI + exact paired sign test) was *built* for the n?6 regime ? which **is** the per-user regime. Promotion-only-on-lift (CI excludes 0 **and** sign test agrees) is precisely *"turn a cue ON for a user only if it helps that user."*
48   - **Replayable feature substrate.** `fusion_golden.py` writes one small JSON per video; GPU touches each video once; re-scoring any variant later is pure-CPU. Adding the user's 11th video = extract once, re-run LOVO over the grown corpus. This *is* the per-user-corpus-growth loop.
49   - **Per-corpus model fit + pinnable artifact.** `distill_local.run()` fits a final model

on all videos; `LogisticRegression.save/load` makes it a kilobyte JSON ? the natural per-user model file.
- **Cold-start fallback exists.** Label-free **Gate-A (net-crossings ? 2)** is the only currently-*validated* fusion win ? **?F1 +0.074, 6/6 golden matches, sign-test p=0.031, CI [+0.042,+0.104]** ? and needs **zero** per-user labels. Gemini remains an always-ready inference fallback (never a training source).

The gap list ? what the pivot must FUND (the hard, unbuilt 80%)
1. **No serving path for a learned per-user model.** `LogisticRegression.load` is **never called anywhere in `backend/pipeline/`** (verified). Production reads only the Gate thresholds. "Optimize a per-user model" currently produces an artifact **the live pipeline cannot consume**. *Per-user TUNING is serveable; per-user LEARNING is not.* This is the single highest-leverage wall.
2. **Correction-loop ? label write path is a stub.** `human_label` exists as a DB column and a read-side filter (`only_unlabeled`), but **nothing in production ever SETS it** (verified ? no setter, no reject/confirm API in `main.py`). The pivot's most load-bearing assumption is the **least built**.
3. **No user/tenant dimension.** No `user_id`/tenant, no per-user model store, no per-user versioning, no per-user leaderboard partition, no per-user regression baseline. Harness is explicitly **"single-tenant / local-first ? NOT MLflow/W&B/a service"** (EXPERIMENT_HARNESS §9).
4. **No continual/automatic re-fit.** `LogisticRegression.fit` is a full batch refit via a CLI. "Continually self-optimizes" today = "an operator re-runs a command."
5. **Per-user label de-biasing.** LOVO consumes golden/silver labels; noisy partial user corrections (false-positives ? misses) need a recall-protecting prior / active-learning prompts ? unbuilt.
6. **Licensing seam.** The only currently-wired training-label source is **Gemini-silver (`distill_local.py`) ? forbidden as a training source** (OWNED_MODEL M0.3(i), a live violation to purge). The correction loop is the clean replacement on this path, but must be built on the human-correction side.

Owned-model roadmap fit. The cheap per-user **decision** layer (thresholds + LogReg + cue ON/OFF + ~1.1M-param Gen-0 head) is feasible-now and roadmap-compatible. The heavy **Qwen3-VL** layer is **not**: on-device fine-tune needs rented 24?48 GB ("cannot QLoRA on the 8 GB box"); even 4B inference latency at 1080p/60fps is an **explicit unmeasured open risk**; budget is the "largest open risk, currently unquantified." On-device is explicitly **"LATER ? don't over-index on it now."**

---

4. What this changes about future choices

- **Owned-model architecture.** Forces an explicit **two-layer decoupling**: (1) cheap per-user *decision* personalization (thresholds + LogReg + cue ON/OFF + Gen-0 head) = the **v1** of the pivot, feasible-now; (2) per-user/heavy **VLM** personalization = needs-research, cloud-bound, gated behind the same Phase-2 budget benchmarks already flagged ? and **restricted to archetype (b)** with serious hardware. The owner's "continually self-optimizes" **must mean the cheap layer for v1**, or the claim is infeasible.
- **Data strategy.** Bifurcates into **two governed corpora**: (1) the **BASELINE "batteries"** corpus ? licensed/owned + **Roora-graded**, governed centrally (consent ledger / minors-exclusion / split-by-match); Roora's role *sharpens* from per-user oracle (never scaled ? can't vendor private footage) to **baseline-corpus oracle + cold-start seeder**. (2) **PER-USER** corpora ? own footage + correction-loop labels, single per-user opt-in, ideally never leaving the device. **Honest correction:** the correction loop feeds *personalization ~90%* and the *shared baseline ~10%* (only via a federated-coefficient nudge); the real baseline-upgrade engine is **seed + opt-in + synthetic**, and the **"<5 videos" benchmark it optimizes is a held-out-USER = GENERALIZATION metric.** Roora spend does **not** shrink ? the baseline still needs an independent vendor-graded golden set to certify it.
- **"Batteries upgrade" mechanism.** Distinguish **MINOR** (feature-schema-stable ? ship new weights, carry each user's per-user LogReg + thresholds + cue_flags **byte-identically**; rollback = config flip on the pinned CONTROL variant) vs **MAJOR** (schema change ? e.g. the planned **M0.4 ActionFormer swap that abandons the 5-feature LogReg** ? every per-user model is dimensionally invalid and **must be re-fit** from stored correction labels; that migration **does not exist**). Upgrade sources by privacy cost: **(1) seed/Roora corpus growth** (do now), **(2) train-only synthetic trajectory augmentation** (must be match-split-clean and *validated to move the HARD multi-court folds* or it's theater), **(3) pooled opt-in** (needs an *enforced*

consent ledger), **(4) federated coefficient averaging** (needs DP noise + minimum-cohort floor + counsel sign-off ? a 6-dim vector over few users is near a per-user fingerprint).
- **Product.** Local install, privacy-first **positioning** ? but **sequenced honestly**: do **not** advertise "data never leaves your machine" until the on-device milestone ships. Cold-start becomes the **#1 product problem**; the correction loop becomes the **central onboarding UX** ("your model just got better"), not a back-end label source. Productize per-user A/B as a **trust feature** ("we tested the motion cue on YOUR videos and it didn't help, so it's off") ? gated behind enough of the user's own labels.
- **Eval.** Two new north-stars: **per-user LOVO** (with a hard minimum-N floor) and a **held-out-USER learning-curve benchmark** ? for a new user's k videos, sweep k=1..K, compare battery v_old vs v_new with the *same* honest ?F1 CI + sign test. "<5 videos to superior quality" becomes the smallest k where held-out F1@tIoU0.5 crosses the ship target (?0.70 multi-court) with CI clearing 0 ? a **falsifiable release criterion**, not a slogan. Add a **per-user no-regression guard** (none exists; the shared-golden gate has no analogue for a deliberately-overfit per-user model).
- **Business model.** One-time install (batteries baseline) + a **"batteries-upgrade" subscription**, kept **under the ~$180/yr cloud anchor** for the hobbyist; a higher **"pro" tier** for archetype (b) (deeper per-user analytics, larger corpora, faster on-device). Validate with a **fake-door + 5?10 buyer interviews re-targeted at individuals** (the research already prescribes this).

---

5. Honest risks + mitigations

Risk	Severity	Mitigation
Small-N per-user eval noise. Per-user A/B is the n?6 regime, *worse*. Verified: the **two recorded fusion A/Bs are both non-significant ? person-fusion 2/6, p=0.6875, ?F1 CI [?0.165,+0.162] (a net LOSS); audio 4/6, p=0.6875, CI [?0.054,+0.203] (crosses 0)**. The first A/B's learned variant scored **F1=0.000** on the 6-rally video.	**HIGH (central)**	**Per-user promotion gate:** a cue/variant flips ON only when ?F1 CI excludes 0 **AND** the sign test agrees on THAT user's held-out videos, with a **minimum-N floor** (refuse learned promotion below n?5; fall back to label-free Gate-A/B). Never disable a **structural** guard (Gate-A held-shuttle defense) on "no lift" when the user's corpus simply lacks that failure mode. **Run the highest-value experiment first:** within-user LOVO (re-run `ab_n6_lovo.py` within-user) to measure whether homogeneous-corpus lower variance beats fewer folds ? *the pivot's core statistical premise is currently unconfirmed.*
Cold-start chasm. Value at 10?15 videos vs incumbents' clip #1. Even rally F1 is only **0.307 shuttle-only / 0.423 learned at n=6**; the multi-court ship target 0.66 is what *Gemini itself* barely reaches.	**HIGH**	Ship the **label-free Gate-A/B fusion** as the day-0 "batteries" (validated, zero-label, +0.074/6-of-6/p=0.031). Frame personalization as *"gets better as it learns YOU."* Instrument **time-to-first-value** and benchmark **video-#1 quality vs cloud incumbents** before committing.
Shot taxonomy. Greenfield + class imbalance + no shot labels from the rally-level correction loop + noisier labels (ROORA flags shot-type needs its own IAA check).	**HIGH**	**Phase it.** Phase 0: rally + highlight + rally-level correction loop. Phase 1: coarse hard/soft proxy from existing speed/arc/reversal signals, default-OFF, measured on golden. Phase 2: finer per-shot UI + Tier-B Roora labels ? **generalization-first** shipped baseline. Phase 3: per-user shot calibration **only for archetype (b)** with enough held-out matches to LOVO-prove a lift. **Do not promise per-user shot tuning to hobbyists** (unfalsifiable for that archetype).
Compute / privacy deferral. VLM fine-tune infeasible on hobbyist hardware; on-device deferred ? near-term personalization trains on **cloud-resident** per-user video, re-opening DPA/retention/Art.9. N per-user fine-tunes = N copies of every governance problem; **unlearning** a "batteries-upgraded" baseline = full retrain (undefined).	**HIGH**	Cap v1 at the **cheap layer** (cloud-free, governance-light). Keep VLM personalization gated behind measured budget/latency benchmarks and **archetype (b) only**. **Move on-device UP the priority list** specifically because it is what makes the privacy claim true. Define a **per-user delete-and-retrain** guarantee; do not pool to the baseline without an *enforced* opt-in + minors-exclusion path.
Per-user overfit traps the test. Invariant features (built so "raw coords don't memorize this court") **cap** the per-user upside *and* a deliberately-overfit model **has no shared baseline to regress against**.	**MED-HIGH**	Keep the invariance discipline (it bounds the downside too); accept that the *real* personalization win is **cue ON/OFF + threshold

selection**, not deep court-overfit. Build a **per-user regression baseline** if any learned per-user model ships. |
84 | **Roora spend doesn't shrink.** Certifying the baseline on biased per-user signal is dishonest; the baseline still needs an independent vendor-graded golden set. | **MED** | Re-scope Roora to (a) certify each batteries upgrade against a held-out generic golden set, (b) seed cold-start labels. **Budget the generic golden set explicitly** ? the "just seed the baseline" framing understates cost. |
85
86   ---
87
88   ## 6. Recommendation ? **HYBRID (adopt-with-conditions): generalization-first baseline + a thin per-user TUNING layer on top**
89
90   **Reject** personalization-first *as the company strategy*. **Adopt** personalization *as a flag-gated, default-honest feature* on a generalization-first baseline.
91
92   **Reasoning.** The pivot is intellectually seductive and the harness *genuinely* makes per-user **tuning** cheap ? that part is real and worth shipping. But as a *strategy* it fails on the project's own honest evidence:
93
94   1. **It auto-promotes config in exactly the eval regime the team's own tooling exists to REFUSE promotion in.** Every comparison at n=6 is non-significant (verified leaderboard: p=0.6875 on both fusion A/Bs, CIs cross/below zero; Gen-0 ship gate refuses at 5/6, p=0.109). Sharding data per-user makes this strictly worse. **When you are data-starved, you pool ? you don't shard.**
95   2. **It inverts the validated market** (away from B2B academies, the #1 beachhead) and **abandons the only compounding moat** (the cross-user dataset/flywheel) for a non-compounding per-seat tool.
96   3. **Its headline KPI ("<5 videos to superior quality") is a held-out-USER = GENERALIZATION metric** ? improving it *is* the generalization-first objective wearing a personalization hat. You cannot honestly certify the baseline on biased per-user signal.
97   4. **Its privacy headline is deferred** to an unbuilt on-device milestone; near-term it re-incurs the cloud governance surface it claims to escape.
98   5. **The differentiated thing is the unbuilt thing.** The harness supports per-user *tuning* (near-ready) far better than per-user *learning* (no serving path, no label write path, no tenant dimension). "Designed for this" conflates "the easy 20% is done" with "the hard 80% is implied."
99
100   **What the hybrid captures (the real, cheap value):**
101   - Day-0 **batteries** = the validated **label-free Gate-A/B** fusion (ships now, zero per-user labels).
102   - A **thin per-user tuning layer**: per-user cue ON/OFF + threshold selection, gated behind the honest ?F1 sign-test with a **hard minimum-N floor**, falling back to the shipped baseline below it. Surfaced as a **trust feature**, not a faith claim.
103   - The **cross-user data flywheel stays alive** (consented data keeps improving the shared "batteries"); per-user tuning rides on top.
104   - Heavy VLM/on-device personalization stays **gated and archetype-(b)-scoped** until budget/latency are *measured*.
105
106   This is the version that does **not** bet the company on per-user overfitting, small-N auto-promotion, on-device VLM fine-tuning, or a shattered flywheel ? while keeping everything the harness genuinely makes cheap.
107
108   ---
109
110   ## 7. Decision points this forces the owner to make NEXT
111
112   1. **Strategy vs feature.** Is personalization **the strategy** (re-run the PMF market-of-record under local+individual assumptions; re-pick the beachhead away from academies) ? or **a feature** on a generalization-first baseline? *Recommendation: feature.* **This is the gating decision; everything below depends on it.**
113   2. **Which moat.** Cross-user **data flywheel** (platform) or per-user **switching-cost** (premium tool)? The strategy docs assume the former; the pivot undercuts it. If platform, a hybrid is mandatory; if tool, drop the cross-user-moat framing to stop relitigating it.
114   3. **Per-user TUNING vs LEARNING ? and the minimum-N floor.** Commit to per-user

config (serveable now) vs per-user *learned models* (needs the serving bridge + correction-loop write path + per-user store ? the funded gap list). Set the **minimum user-corpus N** below which per-user auto-promotion is refused and the system falls back to label-free Gate-A/B.
115   4. **Beachhead archetype.** Lead with the **corpus-rich professional (b)** (personalization value scales with corpus + WTP) or keep **academies (B2B)**? The hobbyist (a) is, on the evidence, a **funnel, not the primary payer**.
116   5. **Privacy sequencing.** Do **not** advertise "data never leaves your machine" until on-device ships. Decide whether to **move on-device UP the roadmap** (it is what makes the privacy claim true) or accept that near-term personalization carries the full cloud DPA/Art.9 posture.
117   6. **Highest-value experiment, fund it first.** Authorize the **within-user LOVO** measurement (re-run `ab_n6_lovo.py` restricted to within-user splits on the GPU/WSL box) to confirm-or-kill the pivot's core statistical premise (does within-user calibration transfer better than cross-video?) **before** funding any per-user serving + storage build-out.
118
119   ---
120
121   ## Evidence Appendix (verified this session ? absolute paths)
122
123   - `C:\Users\avidu\Projects\badminton-highlight-indexer\eval_baselines\experiment_leaderboard.json` ? the two recorded fusion A/Bs: **shuttle_person_fusion 2/6, p=0.6875, ?F1 CI [?0.165,+0.162] (net LOSS)**; **shuttle_person_audio 4/6, p=0.6875, CI [?0.054,+0.203] (crosses 0)**. The empirical heart of the small-N risk. (Now tracked via PR #139.)
124   - `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md` ? shuttle-only **0.307**, learned+threshold **0.423**, **Gemini wrapper 0.66 ship target**; **Gate-A +0.074, 6/6 matches, sign-test p=0.031, CI [+0.042,+0.104]**; standing lesson *"Calibration, not representation, is the small-N wall ? more distinct matches is the highest-ROI unblock."*
125   - `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\` ? **`LogisticRegression.load` is never called here** (grep verified): the per-user learned-model serving gap is real.
126   - `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py` (lines ~110, ~299) ? `human_label` exists as a **column + read-side `only_unlabeled` filter only**; **no production writer** ? the correction-loop write path is a stub.
127   - `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\sports\badminton.py` ? `smash/clear/drop/drive/serve` appear **only in the labeling guideline string**; **no shot-type detection code exists** (shot taxonomy is greenfield).
128   - `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\EXPERIMENT_HARNESS.md` (§9, line ~209) ? *"Single-tenant / local-first ? NOT MLflow/W&B/a service"* (the "designed for generalization, not per-user" evidence).
129   - `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md` (lines ~19?20, ~148) ? *"On-device is LATER ? don't over-index on it now"*; VLM budget *"largest open risk, currently unquantified."*
130

4. user (2026-06-14T08:52:40.191Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
```

```
    (per-frame signals ? run once on the GPU/WSL box, persisted into
    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
    given persisted candidate features it produces `List[Interval]` deterministically,
    with no GPU and zero production risk. So most experiments are pure offline
    re-scoring of stored data and never touch the served pipeline.

    Three modes (`EXPERIMENT_HARNESS.md` §5):
      - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
        held-out loop, but variant-parameterised.
      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
        human spot-checks (and what two highlight reels would show side by side).
      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
        (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

```

```
        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
```

```
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
```

```
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
```

```
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

```

```
        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all
videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
```

```
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376 
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382 
383 
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387 
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                              "variant_b": _segmentation_block(videos, eval_b)},
400         }
401 
402 
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407 
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411 
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422 
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
```

```
m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured
against
        # different labels" the same way regression.gt_hash guards the no-regression
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
```

```
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
```

```
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

5. assistant (2026-06-14T08:52:50.991Z)

Note: the task asks me to write the design to `scratch/PERSONALIZATION_FEATURE_DESIGN.md`, but I'm operating in READ-ONLY planning mode and cannot create files. I'll deliver the full design as my response text. Let me continue exploring the real code to ground the plan.

6. user (2026-06-14T08:52:51.208Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4       further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5       using a few resolution/fps-INVARIANT local features, so the model transfers across
6       videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
```

```
    from backend.eval.metrics import score
    from backend.eval.gemini_refine import merge_overlaps

    Interval = Tuple[float, float]

    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
    # Recall-oriented proposal: deliberately permissive so real rallies are among the
    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
    PROPOSAL = (0.005, 1.0, 0.5)
    BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning


    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
        X: List[List[float]] = []
        for w, g in zip(wins, gated):
            dur = max(1e-6, w["end"] - w["start"])
            win_frames = max(1.0, dur * fps)
            X.append([
                dur,                                          # rally length (sec) ?
fps-invariant
                float(w["peak_velocity"]),                    # already normalized by
frame_width
                float(w["mean_velocity"]),
                float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ?
fps-invariant
                float(g.crossings),                           # net crossings (count) ?
the rally signal
            ])
        return intervals, X


    def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = BASELINE_LOCAL
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        return [(w["start"], w["end"]) for w in wins]


    def load_video(spec: Dict, iou: float = 0.5) -> Dict:
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        intervals, X = build_candidates(spec["trajectory"], fps, fw)
        gts = load_golden_gts(spec["labels"])
        y = label_candidates(intervals, gts, iou)
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}


```

```
    def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals: List[Interval],
                            threshold: float = 0.5) -> List[Interval]:
        if not intervals:
            return []
        proba = model.predict_proba(X)
        pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
        return merge_overlaps(pos, gap_tol=1.0)


    def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
            model_out: Optional[str] = None) -> dict:
        videos = [load_video(s, iou) for s in specs]
        print(f"=== Distilled local rally classifier vs Gemini silver-GT "
              f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
        for v in videos:
            ceil = score(v["intervals"], v["gts"], iou).recall
            print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
                  f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

        result: dict = {"videos": [v["name"] for v in videos]}
        if len(videos) >= 2:
            print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-out) ---")
            clf, base = [], []
            for i, held in enumerate(videos):
                others = [v for j, v in enumerate(videos) if j != i]
                X = [row for v in others for row in v["X"]]
                y = [t for v in others for t in v["y"]]
                model = LogisticRegression().fit(X, y, FEATURE_NAMES)
                preds = predict_rallies(model, held["X"], held["intervals"], threshold)
                sc = score(preds, held["gts"], iou)
                bsc = score(held["baseline"], held["gts"], iou)
                clf.append(sc)
                base.append(bsc)
                print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f} R={sc.recall:.3f} F1={sc.f1:.3f}"
                      f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f} F1={bsc.f1:.3f}")
            def mean(xs, a):
                return sum(getattr(s, a) for s in xs) / len(xs)
            print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f} R={mean(clf,'recall'):.3f} "
                  f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
                  f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
            result["mean_learned_f1"] = mean(clf, "f1")
            result["mean_baseline_f1"] = mean(base, "f1")

        # Final model on ALL videos (for production / running on a fresh held-out video).
        Xall = [row for v in videos for row in v["X"]]
        yall = [t for v in videos for t in v["y"]]
        final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
        assert final.w is not None  # set by fit()
        weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
        print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
        if model_out:
            final.save(model_out)
            print(f"final model (all videos) -> {model_out}")
            result["model_out"] = model_out
        return result


    def main() -> None:
        ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally classifier.")
```

```
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161
```

7. user (2026-06-14T08:52:51.816Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
```

```
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
```

```
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
            (0) keep every window ? identical to the substrate. Persisted candidates are
            untouched (full superset)."""
            fcfg = idx_cfg.get("fusion", {})
            min_crossings = int(fcfg.get("min_crossings", 0))
            min_players = int(fcfg.get("min_players", 0))
            person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
```

```
143
144             keep: List[int] = []
145             for i, st in enumerate(window_stats):
146                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                     continue
148                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                     continue
150                 keep.append(i)
151             return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

8. assistant (2026-06-14T08:53:00.721Z)

Now let me read the config, database, main API, and eval_stats files.

9. user (2026-06-14T08:53:01.564Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
```

```
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
        # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
        min_free_gb: float = 0.0


    class FusionConfig(BaseModel):
        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).

        Every default reproduces control behaviour: the fusion segmenter persists the
        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
        true ? the person-cue features, but it does NOT gate the served handoff unless a
        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
        supplied ? off-court persons (spectators / adjacent courts) are excluded.
        """
        # Which trajectory substrate seeds the windows (shuttle stays primary).
        substrate: Literal["wasb", "tracknet"] = "wasb"
        # Windowing velocity threshold for the fusion family (the engine reads
        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
        velocity_threshold: float = 0.02
        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
        # enabled ? so the default path never imports torch / touches the GPU.
        cue_flags: dict[str, bool] = Field(default_factory=dict)
        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
```

```
        min_crossings: int = Field(default=0, ge=0)
        # Served Gate B: when the person cue is on, drop windows with median in-court
players
        # < this. 0 = OFF.
        min_players: int = Field(default=0, ge=0)
        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
        court_polygon: Optional[list[list[float]]] = None
        # Net line for the person two-sided-motion split (person features only ? the shuttle
        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
        net_axis: Literal["x", "y"] = "y"
        net_position: float = Field(default=0.5, ge=0.0, le=1.0)


    class IndexingConfig(BaseModel):
        default_segmenter: str = "yolo_hybrid"
        use_gpu: bool = True
        motion_threshold: float = 0.08
        min_segment_duration: float = 3.0
        dead_time_merge_gap: float = 2.0
        chunk_duration: float = 90.0
        chunk_overlap: float = 10.0
        detector_model: str = "fasterrcnn_resnet50_fpn"
        detector_score_threshold: float = 0.5
        yolo_velocity_threshold: float = 0.15
        yolo_frame_skip: int = 3
        yolo_tracker: str = "bytetrack.yaml"
        yolo_prefetch_workers: int = 2
        min_action_window_duration: float = 2.0
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
```

```
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]

```

```
222
223     class AppConfig(BaseModel):
224         """Root configuration object.
225
226         All runtime configuration should be accessed via an instance of this class
227         (or the loader) rather than raw dict lookups.
228         """
229
230         sport: Sport = "badminton"
231         ai_provider: str = "gemini"
232         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
233         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
234         ai_model: str = "gemini-flash-latest"
235
236         validation: ValidationConfig = Field(default_factory=ValidationConfig)
237         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
238         output: OutputConfig = Field(default_factory=OutputConfig)
239         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
240         security: SecurityConfig = Field(default_factory=SecurityConfig)
241         logging: LoggingConfig = Field(default_factory=LoggingConfig)
242
243         # Allow extra keys during transition so existing config.json files do not break.
244         model_config = {
245             "extra": "allow",
246             "validate_assignment": True,
247         }
248
249         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
250         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
251         # attribute access / the typed config. A nested *section* is returned as a plain
dict
252         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
253         # section model instead breaks it (a model has no .get), which bit every validator
on
254         # real runs. Scalar leaves are returned as-is.
255         def get(self, key: str, default: Any = None) -> Any:
256             _MISSING = object()
257             val: Any = _MISSING
258             if key in type(self).model_fields or hasattr(self, key):
259                 val = getattr(self, key, _MISSING)
260             if val is _MISSING:
261                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
262                 # discovered dynamically from the model fields, so a future top-level
section
263                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
264                 for fname in type(self).model_fields:
265                     section = getattr(self, fname, None)
266                     if isinstance(section, BaseModel) and hasattr(section, key):
267                         val = getattr(section, key)
268                         break
269             if val is _MISSING:
270                 return default
271             if isinstance(val, BaseModel):
272                 return val.model_dump(mode="json")
273             return val
274
275         def __getitem__(self, key: str) -> Any:
276             val = self.get(key)
277             if val is None and key not in type(self).model_fields:
278                 # Re-raise like a real dict for unknown top-level
```

```
279                 raise KeyError(key)
280             return val
281
282
283     # Convenience type alias for callers that want to annotate
284     Config = AppConfig
285
```

10. user (2026-06-14T08:53:01.947Z)

```
1       """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3       The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4       single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5       and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6       ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7       turns those into an honest summary:
8
9       - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10        interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11        many *videos* it wins on (the distribution-free test the repo already leans on:
12        "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14        producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15        match, never a held-out window from the same video).
16
17      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20      """
21      from __future__ import annotations
22
23      import math
24      import random
25      import statistics
26      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28      Interval = Tuple[float, float]
29      FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                       n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38          """
39          vals = [float(v) for v in values]
40          n = len(vals)
41          if n == 0:
42              return (0.0, 0.0)
43          if n == 1:
44              return (vals[0], vals[0])
45          rng = random.Random(seed)
46          means: List[float] = []
47          for _ in range(max(1, n_boot)):
48              means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
```

```
        means.sort()
        alpha = (1.0 - confidence) / 2.0
        last = len(means) - 1
        lo = means[max(0, int(math.floor(alpha * last)))]
        hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
        return (lo, hi)


    def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
        """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
        vals = [float(v) for v in values]
        n = len(vals)
        mean = statistics.mean(vals) if vals else 0.0
        std = statistics.pstdev(vals) if n > 1 else 0.0
        lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
        return {"mean": mean, "std": std, "n": float(n),
                "ci_low": lo, "ci_high": hi, "confidence": confidence}


    def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
        """Two-sided exact sign test over per-fold deltas (B ? A).

        ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
        excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
        distribution-free "did B win on enough *videos*?" check.
        """
        wins = sum(1 for d in deltas if d > eps)
        losses = sum(1 for d in deltas if d < -eps)
        ties = len(deltas) - wins - losses
        n_eff = wins + losses
        if n_eff == 0:
            p = 1.0
        else:
            k = min(wins, losses)
            tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
            p = min(1.0, 2.0 * tail)
        return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
                "n_effective": float(n_eff), "p_value": p}


    def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                        eps: float = 0.0) -> Dict[str, Any]:
        """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

        ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
        fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
        test agrees ? not when a single mean ticked up.
        """
        n = min(len(a_values), len(b_values))
        deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
        summary = mean_with_ci(deltas, confidence, n_boot, seed)
        return {
            "mean_delta": summary["mean"],
            "ci_low": summary["ci_low"],
            "ci_high": summary["ci_high"],
            "n": float(n),
            "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
            "sign_test": paired_sign_test(deltas, eps),
        }

```

```
111
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150
```

11. assistant (2026-06-14T08:53:04.599Z)

Now let me read the database schema and the main API surface. These are large, so I'll focus on the relevant parts.

12. user (2026-06-14T08:53:05.382Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
```

```
            self.db_path = db_path
            self._init_db()

        @contextmanager
        def _connect(self):
            """Transaction scope that also CLOSES the connection.

            ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
            never closes, so every call leaked its connection to GC (lingering file
            handles on the WAL sidecars under Windows). Internal methods use this.
            """
            conn = self._get_connection()
            try:
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

```

```
                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
```

```
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")
                # future: if version < 4: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 4

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
```

```
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
```

```
236                 return None
237
238         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
239             try:
240                 with self._connect() as conn:
241                     conn.cursor().execute(
242                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
243                         (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
244                     )
245                     conn.commit()
246                 return True
247             except Exception as e:
248                 logger.error(f"Failed to add event: {e}")
249                 return False
250
251         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
252                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
253                                   stats: Optional[Dict[str, Any]] = None,
254                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
255             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
256             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
257             try:
258                 with self._connect() as conn:
259                     cursor = conn.cursor()
260                     cursor.execute(
261                         """INSERT INTO candidate_segments
262                            (video_id, detector, chunk_index, start_time, end_time, duration,
263                             confidence, stats, detection_params)
264                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
265                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
266                          confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
267                     )
268                     conn.commit()
269                     return cursor.lastrowid
270             except Exception as e:
271                 logger.error(f"Failed to add candidate segment: {e}")
272                 return None
273
274         def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
275             """Record that a candidate detection was confirmed as a given play_segment."""
276             try:
277                 with self._connect() as conn:
278                     conn.cursor().execute(
279                         "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
280                         (segment_id, candidate_id)
281                     )
282                     conn.commit()
283                 return True
284             except Exception as e:
285                 logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
286                 return False
287
288         def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
289                                      only_unlabeled: bool = False) -> List[Dict[str, Any]]:
```

```
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
```

```
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
                        "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                        (job_id, video_path, segmenter,
                         json.dumps(player_pool) if player_pool is not None else None,
                         max_frames, json.dumps(policy) if policy is not None else None)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to create job {job_id}: {e}")
                return False

        def mark_job_running(self, job_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                        "progress='starting' WHERE id = ?", (job_id,))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} running: {e}")
                return False

```

```
        def set_job_progress(self, job_id: str, progress: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set video_id for job {job_id}: {e}")
                return False

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='done', progress='finished', result=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                        (json.dumps(result), job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} done: {e}")
                return False

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='failed', error=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} failed: {e}")
                return False

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

```

```
476         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
477         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
478             """Attach/replace a job's JSON ExecutionPolicy."""
479             try:
480                 with self._connect() as conn:
481                     conn.cursor().execute(
482                         "UPDATE jobs SET policy=? WHERE id = ?",
483                         (json.dumps(policy) if policy is not None else None, job_id))
484                     conn.commit()
485                 return True
486             except Exception as e:
487                 logger.error(f"Failed to set policy for job {job_id}: {e}")
488                 return False
489
490         def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
491             """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
492             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
493             computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
494             try:
495                 with self._connect() as conn:
496                     conn.cursor().execute(
497                         "UPDATE jobs SET status='waiting', "
498                         "next_poll_at=datetime('now', ?), "
499                         "progress=COALESCE(?, progress) WHERE id = ?",
500                         (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
501                     conn.commit()
502                 return True
503             except Exception as e:
504                 logger.error(f"Failed to set job {job_id} waiting: {e}")
505                 return False
506
507         def claim_due_job(self) -> Optional[Dict[str, Any]]:
508             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
509             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
510             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
511             first. Returns the claimed job dict, or None if nothing is runnable."""
512             try:
513                 with self._connect() as conn:
514                     cur = conn.cursor()
515                     row = cur.execute(
516                         "SELECT id, status FROM jobs WHERE status='queued' "
517                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
518                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
519                     if not row:
520                         return None
521                     cur.execute(
522                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
523                         "WHERE id=? AND status=?", (row["id"], row["status"]))
524                     conn.commit()
525                     if cur.rowcount != 1:
526                         return None                     # lost the race to a concurrent
worker
527                 return self.get_job(row["id"])
528             except Exception as e:
529                 logger.error(f"Failed to claim a due job: {e}")
```

```
530                 return None
531
532         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
533             """Crash recovery: any job still queued/running from a previous process is dead
534             (the executor is in-process). Mark failed so clients see a terminal state, not a
535             hang. Called once at startup. Returns the number of jobs swept."""
536             try:
537                 with self._connect() as conn:
538                     cur = conn.cursor()
539                     cur.execute(
540                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
541                         "WHERE status IN ('queued', 'running')", (reason,))
542                     conn.commit()
543                     return cur.rowcount
544             except Exception as e:
545                 logger.error(f"Failed to sweep stale jobs: {e}")
546                 return 0
547
```

13. assistant (2026-06-14T08:53:08.377Z)

Now let me look at the main.py API surface, the branch for the per-user promotion gate, and the rally-annotator sibling. Let me check git state and these files in parallel.

14. user (2026-06-14T08:53:10.538Z)

```
+ feat/per-user-promotion-gate
* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/chore/gitignore-scratch
  remotes/origin/chore/scripts-dir-and-layout-guidelines
  remotes/origin/chore/track-leaderboard
  remotes/origin/ci/ruff-lint-lane
  remotes/origin/claude/docs-handoff-consistency
  remotes/origin/claude/highlight-watermark
  remotes/origin/claude/indexer-doc-review-XVPTh
  remotes/origin/claude/platform-architecture-research
  remotes/origin/claude/watermark-bundled-font
  remotes/origin/docs/analyzers-and-reconcilers
  remotes/origin/docs/analyzers-framework
  remotes/origin/docs/execution-policy
  remotes/origin/docs/first-ab-experiment
  remotes/origin/docs/gate-a-promotion-and-layout
  remotes/origin/docs/i3-fusion-measured
  remotes/origin/docs/next-steps-multisport
  remotes/origin/docs/pmf-field-signals
  remotes/origin/docs/storage-sharing-model
  remotes/origin/docs/video-locality-model
  remotes/origin/feat/async-job-model
  remotes/origin/feat/eval-honesty-corrections
  remotes/origin/feat/execution-policy-p1
  remotes/origin/feat/execution-policy-p2
  remotes/origin/feat/fusion-extraction-speedup
  remotes/origin/feat/fusion-honest-delta
  remotes/origin/feat/fusion-observability
  remotes/origin/feat/fusion-p3
  remotes/origin/feat/fusion-p4-audio
  remotes/origin/feat/fusion-segmenter
  remotes/origin/feat/fusion-smallest-first
  remotes/origin/feat/harness-honest-reporting
  remotes/origin/feat/khelacharya-shot-intelligence
  remotes/origin/feat/learned-variant-fix
  remotes/origin/feat/owned-model-study-and-prototype
  remotes/origin/feat/run-telemetry
```

```
  remotes/origin/feat/telemetry-wire-fusion-golden
  remotes/origin/fix/stitching-padding-config
---CURRENT---
master
```

15. user (2026-06-14T08:53:11.135Z)

```
770 backend/main.py
```

16. assistant (2026-06-14T08:53:17.383Z)

```
The branch `feat/per-user-promotion-gate` exists locally. Let me read the main.py API surface,
examine the per-user-promotion-gate branch diff, and check the rally-annotator sibling project.
```

17. user (2026-06-14T08:53:18.347Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import threading
6       import traceback
7       import uuid
8       from concurrent.futures import ThreadPoolExecutor
9       import uvicorn
10      from dotenv import load_dotenv
11
12      load_dotenv()
13
14      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15      logging.basicConfig(
16          level=logging.INFO,
17          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18          datefmt='%H:%M:%S'
19      )
20      logger = logging.getLogger(__name__)
21      from fastapi import FastAPI, HTTPException, Request
22      from fastapi.middleware.cors import CORSMiddleware
23      from pydantic import BaseModel
24      from typing import List, Dict, Any, Optional
25
26      from fastapi.staticfiles import StaticFiles
27      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28      from backend.pipeline.validators.base import registry
29      from backend.infrastructure.database import Database
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.stitcher.compiler import VideoCompiler
32      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
33      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
35      from backend.results import Failure  # standardized error/result contract
36      from backend.reporting import emit_indexer_report
37
38      app = FastAPI(title="Sports Highlight Indexer API")
39
40      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
41      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43      app.add_middleware(
44          CORSMiddleware,
```

```
45          allow_origins=["*"],
46          allow_credentials=False,
47          allow_methods=["*"],
48          allow_headers=["*"],
49      )
50
51      # Serves static frontend files directly (now under api/ per approved structure)
52      os.makedirs("backend/api/static", exist_ok=True)
53      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55      @app.get("/", response_class=HTMLResponse)
56      def serve_index():
57          index_path = "backend/api/static/index.html"
58          if os.path.exists(index_path):
59              with open(index_path, "r", encoding="utf-8") as f:
60                  return f.read()
61          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63      # Global variables initialized at startup
64      db_instance: Optional[Database] = None
65      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
66
67      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70      # already parallelize their AI handoff internally via their own pools.
71      _job_executor: Optional[ThreadPoolExecutor] = None
72
73      def _get_executor() -> ThreadPoolExecutor:
74          """Lazy module-global executor; tests monkeypatch this for inline execution."""
75          global _job_executor
76          if _job_executor is None:
77              _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78          return _job_executor
79
80      # Legacy thin wrapper for any remaining direct calls during transition.
81      # New code should import load_app_config / AppConfig from backend.config directly.
82      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83          cfg = load_app_config(config_path)
84          return cfg.model_dump(mode="json")
85
86      # --- API Models ---
87      class ProcessRequest(BaseModel):
88          video_path: str
89          player_pool: Optional[List[str]] = None
90          max_frames: Optional[int] = None
91          segmenter: Optional[str] = None
92
93      class QueryRequest(BaseModel):
94          video_id: str
95          server: Optional[str] = None
96          receiver: Optional[str] = None
97          duration_min: Optional[float] = None
98          event_type: Optional[str] = None
99
100     class CompileRequest(BaseModel):
101         video_id: str
102         segment_ids: List[int]
103         output_filename: str
104
105     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
```

```
        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
```

```
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
```

```
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    def _run_job(job_id: str, req: ProcessRequest) -> None:
        """Job worker: drives _execute_processing and records terminal state.

        Job `status` is the EXECUTION state (queued|running|done|failed): 'done' means the
        worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]. 'failed' means the worker itself crashed/raised.
        """
        db_instance.mark_job_running(job_id)
        try:
            result = _execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
            if result.get("video_id"):
                db_instance.set_job_video_id(job_id, result["video_id"])
            db_instance.mark_job_done(job_id, result)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_run_job, job_id, req)
        return JSONResponse(
```

```
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    # proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
    # under security.upload_dir, then submitted to /api/process by the returned path.
    # Upload state is in-process by design (this is the single-box alpha): a server restart
    # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    # arrives with PR-3 identity scoping.

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        sec = getattr(global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass
```

```
349
350
351     @app.post("/api/upload/init")
352     def upload_init(req: UploadInitRequest):
353         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
354         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
355         try:
356             name = safe_output_filename(req.filename)
357         except ValueError as e:
358             raise HTTPException(status_code=400, detail=str(e)) from e
359         ext = os.path.splitext(name)[1].lower().lstrip(".")
360         if ext not in exts:
361             raise HTTPException(status_code=400,
362                                 detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
363         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
364             raise HTTPException(status_code=413,
365                                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
366         os.makedirs(upload_dir, exist_ok=True)
367         upload_id = uuid.uuid4().hex
368         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
369         open(tmp_path, "wb").close()
370         with _uploads_lock:
371             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
372         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
373                 "next_part": f"/api/upload/{upload_id}/part/0"}
374
375
376     @app.put("/api/upload/{upload_id}/part/{index}")
377     async def upload_part(upload_id: str, index: int, request: Request):
378         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
379         _, max_bytes, part_bytes, _ = _upload_cfg()
380         with _uploads_lock:
381             st = _uploads.get(upload_id)
382         if st is None:
383             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
384         if index != st["parts"]:
385             raise HTTPException(status_code=409,
386                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
387
388         received = 0
389         overflow = None
390         with open(st["tmp_path"], "ab") as fh:
391             async for chunk in request.stream():
392                 received += len(chunk)
393                 if received > part_bytes:
394                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
395                     break
396                 if st["bytes"] + received > max_bytes:
397                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
398                     break
399                 fh.write(chunk)
400         if overflow:
401             _abort_upload(upload_id)
402             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
403
404         with _uploads_lock:
405             st["parts"] += 1
406             st["bytes"] += received
```

```
        return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


    @app.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(status_code=409,
                                detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
            stem, ext = os.path.splitext(st["filename"])
            final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
        os.replace(st["tmp_path"], final_path)
        with _uploads_lock:
            _uploads.pop(upload_id, None)
        video_id = compute_video_id(final_path)
        return {"path": os.path.abspath(final_path), "video_id": video_id,
                "size_bytes": os.path.getsize(final_path),
                "next": "POST /api/process with this path"}


    @app.delete("/api/upload/{upload_id}")
    def upload_abort(upload_id: str):
        """Abort an in-flight upload and delete its partial file."""
        with _uploads_lock:
            known = upload_id in _uploads
        if not known:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        _abort_upload(upload_id)
        return {"status": "aborted", "upload_id": upload_id}


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
```

```
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
f"stitching.min_shot_count={stitching.min_shot_count}.")
```

```
524
525         # Extract start/end time pairs
526         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
527
528         # Run compiler with the configured stitching paddings + optional branding watermark
529         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
530         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
531                                  end_padding=stitching.end_padding,
watermark=stitching.watermark)
532         try:
533             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
534             return {"status": "success", "filepath": out_path}
535         except Exception as e:
536             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
537
538     # --- CLI Debug Utility & Orchestration ---
539     def run_cli_diagnose_clip(video_path: str, timestamp: float):
540         """CLI debugging utility to examine segment properties around a timestamp."""
541         print("=" * 60)
542         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
543         print("=" * 60)
544
545         cfg = load_config()  # legacy wrapper still works, returns dict for now
546
547         # 1. Validation check diagnostics
548         print("[DIAGNOSTIC] Running validation framework...")
549         results = registry.run_all(video_path, cfg)
550         for r in results:
551             status_symbol = "[PASS]" if r.passed else "[FAIL]"
552             print(f"  {status_symbol} {r.validator_name}: {r.message}")
553             if r.details:
554                 print(f"         Details: {r.details}")
555
556         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
557         print("-" * 60)
558         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
559         import cv2
560         cap = cv2.VideoCapture(video_path)
561         if not cap.isOpened():
562             print("[FAIL] OpenCV could not open video.")
563             return
564
565         fps = cap.get(cv2.CAP_PROP_FPS)
566         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
567
568         start_frame = max(0, int((timestamp - 3.0) * fps))
569         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
570
571         # Measure frame-by-frame changes in sample region
572         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
573         prev_gray = None
574         motions = []
575
576         for _ in range(start_frame, end_frame):
577             ret, frame = cap.read()
578             if not ret:
579                 break
580             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
581             gray = cv2.GaussianBlur(gray, (21, 21), 0)
582
583             if prev_gray is not None:
584                 diff = cv2.absdiff(prev_gray, gray)
585                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
```

```
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke setup diagnostics
            import subprocess
            print("[INFO] Invoking setup.py...")
            subprocess.run([sys.executable, "setup.py"])
            return

```

```
        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
```

```
695             print(f"[CLI] Processing video file: {args.video_path}")
696             if not os.path.exists(args.video_path):
697                 print(f"[FAIL] Video file not found: {args.video_path}")
698                 return
699 
700             video_id = compute_video_id(args.video_path)
701             was_reingest = db_instance.get_video(video_id) is not None
702 
703             # Validate (with-context variant surfaces ffprobe_meta for the report)
704             print("[CLI] Validating...")
705             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
706             failures = [r for r in val_results if not r.passed]
707 
708             # Save video status
709             status = "failed" if failures else "processed"
710             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
711 
712             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
713             segmenter_actual = None
714             segmentation_ran = False
715             try:
716                 print("\n" + "="*40)
717                 print("VALIDATION SUMMARY")
718                 print("="*40)
719                 for r in val_results:
720                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
721                     print(f"{status_symbol} {r.validator_name}: {r.message}")
722                 print("="*40 + "\n")
723 
724                 if failures:
725                     print("[FAIL] Video failed checks. Indexing halted.")
726                     return
727 
728                 # Index
729                 segmenter_cls = segmenter_registry.get(seg_name)
730                 if not segmenter_cls:
731                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
732                     return
733 
734                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
735                 segmenter_actual = seg_name
736                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
737                 segmenter = segmenter_cls(db_instance, global_config)
738                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
739                 segmentation_ran = True
740 
741                 if isinstance(result, Failure):
742                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
743                     print("[FAIL] Indexing halted for video.")
744                     segments = []
745                 else:
746                     segments = result.value if hasattr(result, "value") else result
747                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
748                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
749                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
750             finally:
751                 # Always emit the per-video observability report. Never raises.
752                 paths = emit_indexer_report(
753                     db=db_instance, video_id=video_id, video_path=args.video_path,
```

```
config=global_config,
754                     val_results=val_results, val_context=val_context,
755                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
756                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
757                 )
758                 if paths:
759                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
760                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
761                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
762             return
763
764         # Start Local Server Mode
765         if args.server or not args.video_path:
766             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
767             uvicorn.run(app, host="127.0.0.1", port=args.port)
768
769     if __name__ == "__main__":
770         main()
771
```

18. assistant (2026-06-14T08:53:22.682Z)

Now let me examine the per-user-promotion-gate branch and the sibling rally-annotator project. Let me look at the branch diff and the classifier (LogisticRegression) save/load.

19. user (2026-06-14T08:53:24.779Z)

===DIFF STAT===

20. user (2026-06-14T08:53:25.122Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
32          @staticmethod
```

```
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

        @classmethod
```

```
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

21. assistant (2026-06-14T08:53:28.945Z)

The branch `feat/per-user-promotion-gate` has no diff against master (it's likely freshly created/empty or pointing at the same commit). Let me confirm and look at the sibling projects and the promotion gate logic that currently exists.

22. user (2026-06-14T08:53:31.127Z)

```
77082f4ccb4766c4773d9899b1bfa806a678058e
77082f4ccb4766c4773d9899b1bfa806a678058e
===merge-base===
77082f4ccb4766c4773d9899b1bfa806a678058e
===log on branch not master===
```

23. user (2026-06-14T08:53:32.227Z)

```
Annotation Setup
CODE WITH ERIC
Email-Mining
KhelSutraGuru.jpeg
Verifiers
badminton-highlight-indexer
bhi-puser
gmail-attachments-to-gdrive
gtest
khelsutra
muneem
rally-annotator
sports-data-collector
sports-obsreport
===rally-annotator search===
6 June 2026
Collect
Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf
Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf
Roora Pilot Review ? Badminton (2026-06-06).md
Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md
golden_labels_badminton.csv
rally_review.csv
```

24. assistant (2026-06-14T08:53:37.310Z)

The `feat/per-user-promotion-gate` branch points at the same commit as master (no work yet ? it's a placeholder to integrate into). There's also a `bhi-puser` sibling (likely "BHI per-

user"). Let me examine the rally-annotator project and the bhi-puser directory, plus the existing promotion gate (regression) and the rally_gate structural guard.

25. user (2026-06-14T08:53:38.935Z)

```
CHANGELOG.md
LICENSE
README.md
docs
test
vlc
===bhi-puser===
CLAUDE.md
README.md
backend
config.json
docs
eval_baselines
mypy.ini
pytest.ini
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
setup.py
tests
tools
training
```

26. user (2026-06-14T08:53:39.001Z)

```
backend\eval\regression.py
```

27. assistant (2026-06-14T08:53:45.441Z)

`bhi-puser` looks like a full sibling clone (possibly the per-user worktree). Let me check whether it has per-user work already, read the rally-annotator README, and read the regression promotion gate.

28. user (2026-06-14T08:53:48.194Z)

```
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
ec5bbe0 chore(eval): track experiment_leaderboard.json (fusion P3/P4 honest LOVO records) (#139)
74c1303 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus (#137)
8836dfe chore: scripts/ for run-drivers + CODE_MAP §0 file-placement guidelines (#138)
fa12829 chore: gitignore scratch/ to keep git status clean (#136)
55df82f feat(obs): run-telemetry black box ? fps + GPU temp/clocks/power + CPU/RAM (rotating
~10KB JSONL) (#134)
a9432a9 feat(db): self-scheduling job substrate ? migration v3 + claim_due_job (P2) (#135)
ec2002d feat(infra): ExecutionPolicy P1 ? bound the Gemini hang (op_timeout + bounded poll)
(#133)
2673eee docs: execution-policy framework ? per-job, self-scheduling resilience (design) (#132)
c8bdffd docs: promote Gate-A (measured win) + per-video output layout plan (#131)
2e5892e feat(eval): fix the learned variant ? train-fold threshold tuning + calibration (C2)
(#130)
f680b8d feat(eval): wire C1+C4 honest reporting into experiment.compare() (additive) (#129)
8398e52 docs(archive): first end-to-end A/B experiment on the golden corpus (n=6) (#128)
d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
813d1f8 feat(eval): fusion_golden --smallest-first (quick-feedback extraction order) (#126)
===BRANCH===
feat/per-user-promotion-gate
```

```
===STATUS===
?? backend/eval/per_user_gate.py
```

29. user (2026-06-14T08:53:48.217Z)

```
1       # Rally Annotator
2
3       [![tests](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml/badge.svg)](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml)
4
5       A tiny **VLC plugin** to hand-label rally **start/end + point-stop reason** while you
watch a match,
6       for **net-separated racquet sports** ? badminton, tennis, table tennis, pickleball,
padel.
7
8       You watch the video in VLC, pause/scrub freely, and click **Mark START** / **Mark END**
(with a
9       reason). It writes one CSV row per rally ? clean ground-truth labels for
training/evaluating rally
10      (point) segmentation models, or just for cutting highlights.
11
12      ```
13      rally_number,start_time,end_time,ending_reason,sport,shots_count
14      1,8.800,11.500,winner,badminton,9
15      2,24.389,46.589,unforced_error,badminton,21
16      ```
17
18      Times are **decimal seconds**; `shots_count` is an **optional** rally shot/stroke count
(blank if you skip it).
19      See [docs/CSV_FORMAT.md](docs/CSV_FORMAT.md).
20
21      ## Why
22      Labeling rally boundaries is the slow, expensive prerequisite for any rally-detection
model. Most tools
23      make you transcribe timestamps by hand. This lets a rater do it **inside the player they
already use**,
24      pausing to the exact frame before each mark ? turning "watch a match" into "produce
golden labels."
25
26      ## Install
27      1. Copy `vlc/rally_annotator.lua` into your VLC Lua **extensions** folder:
28         - **Windows:** `%APPDATA%\vlc\lua\extensions\`
29         - **macOS:** `~/Library/Application Support/org.videolan.vlc/lua/extensions/`
30         - **Linux:** `~/.local/share/vlc/lua/extensions/`
31         (create the `extensions` folder if it doesn't exist)
32      2. In VLC: **Tools ? Plugins and extensions ? Reload extensions** (or just restart VLC).
33      3. Enable it from the **View** menu ? **Rally Annotator**. A small dialog opens and
stays open while you watch.
34
35      Requires **VLC 3.0.x**. (VLC 4.0 changed the Lua input/listener API; targeting 3.x for
now.)
36
37      ## Use
38      **Playback is built in** ? the **Back 5s · Play / Pause · Fwd 5s** row drives the VLC
player from the
39      annotation window, so you can pause, label, and resume without ever switching to the
main VLC window. **Play / Pause**
40      is a single toggle (it resumes when paused, pauses when playing).
41
42      1. Pick the **Sport** (it stays set across rallies).
43      2. When a rally begins, click **Mark START** ? it snapshots the current playback time
into the **Start** field
44         (pause/scrub first for frame accuracy; you can also edit the field by hand).
45      3. When the rally ends, click **Mark END** (fills the **End** field). This *arms* the
```

rally; nothing is written yet.
4. Choose the **Ending reason** (sits between **Mark END** and **Save Rally**; defaults to **`unknown`**), optionally
   type a **Number of shots** (the rally's shot/stroke count ? leave blank to skip), then click
   Save Rally to write one CSV row. The reason **resets to `unknown` after every save** (and the shots field
   clears too) ? so neither silently reuses the previous rally's value, and you can pick the same reason on
   consecutive rallies. Use the **Next rally #** field to resume/insert numbering anywhere.
5. **Recent rallies** list: select a row, then **Edit selected** (loads it back into the fields to fix
   start/end/reason/sport ? **Save changes**) or **Delete selected**. **Undo last** removes the most recent row ?
   the button shows which one, e.g. `Undo last (#7)`, and the status panel shows that row's details ? or clears an
   in-progress mark, or cancels an edit.
6. **Help** button ? usage + an [ending-reason decision guide](docs/ENDING_REASONS.md), rendered inside the dialog.

Output: `<video-stem>.rallies.csv` next to the video (falls back to your home dir if the path can't be
resolved). Re-opening the extension **continues** rally numbering from the existing file ? no duplicate IDs.

Resuming a half-finished video: labels are saved to that CSV as you go. Open the **same video** and enable the
extension and it **reloads your existing rallies** (shown in the Recent list) and continues numbering ? so you can
stop and pick up later. Easiest flow: **open the video first, then enable the extension.** If you switch videos with
the dialog already open, click **Refresh** to point at the current video and load its rallies. (Playback *position*
isn't restored ? scrub to where you stopped.)

> **Where's the help / "About" text?** The empty *"About ?lua"* box under **Tools ? Plugins and extensions ?
> Add-ons** is VLC's own add-on info dialog; its body ("Lua script") is a constant baked into VLC and **can't be
> set by an extension**. Use the **Help** button inside the Rally Annotator dialog instead. (The descriptor's
> description does show in the *Active Extensions* tab's **"More information"** dialog.)

Sports & taxonomy

Net-separated racquet sports share a forced/unforced-error point-stop taxonomy, so one tool covers them all:

Sport	Notes
badminton	the reference sport
tennis	service_fault = fault/double-fault; let supported
table_tennis	let (net serve) supported; fast cadence
pickleball	service_fault for faults; treat "let" per local rules
padel	net-separated; rally boundaries as in tennis

`ending_reason` ? `{unknown, winner, forced_error, unforced_error, service_fault, let, other}` ? a shared
vocabulary across these sports. Keep to these values for clean downstream aggregation.

What the ending reasons mean

`unknown` is the **default** (the field resets to it after every save); every other reason **except `winner`** is
charged to the side that **lost** the rally.

```
89      | Reason | When the rally ended because? |
90      |---|---|
91      | `unknown` | **default** ? not classified yet. A save with no reason picked records
`unknown` (never the previous rally's reason); set a specific reason when you can. |
92      | `winner` | the last shot landed **in** and went unreturned (opponent couldn't reach it
/ only waved). A clean ace counts here. |
93      | `forced_error` | the loser **missed** (out or into the net) while **under pressure** ?
stretched, rushed, jammed, handling the opponent's pace/spin/depth. |
94      | `unforced_error` | the loser **missed a routine shot** they had time **and** position
to make, with little or no pressure. |
95      | `service_fault` | the point ended **on the serve** ? into the net, out of the service
box, illegal action/foot fault, or a double fault. |
96      | `let` | the rally is **replayed** under the rules, no point scored (e.g. a
tennis/table-tennis serve net-cord that's otherwise good, outside interference). |
97      | `other` | none of the above ? occluded footage, injury/retirement, penalty, hindrance.
Use sparingly. |
98
99      **The cases people ask about:** a ball/shuttle landing **out** is the *hitter's* error ?
`forced_error` only if they
100     were under pressure, otherwise `unforced_error` (it's **never** automatically forced,
and never a winner for the
101     opponent). **Into the net** during a rally is the same; a **serve** into the net is
`service_fault`. When unsure
102     forced-vs-unforced, **default to `unforced_error`**. See the full decision guide with
examples and per-sport rules
103     in **[docs/ENDING_REASONS.md](docs/ENDING_REASONS.md)** (also available via the in-
plugin **Help** button).
104
105     ## Roadmap
106     - [ ] Live-test the v1.6 dialog in VLC (single Play / Pause toggle, optional Number of
shots, two-step Save,
107           `unknown`-default reason, editable times, Next rally #, Recent-rallies
Edit/Delete) across all five sports.
108     - [ ] Optional per-sport reason presets / hotkeys.
109     - [ ] Alternative front-ends for power users / remote raters: a `python-vlc` + Tk/Qt app
with global keyboard
110           shortcuts (S/E/1?6/U), and a zero-install HTML5 `<video>` page that exports the
same CSV.
111     - [x] Optional `shots_count` column (v1.6). Further extra columns (e.g. server/receiver)
still TBD behind a toggle.
112
113     ## Tests
114     The dialog logic has a headless test suite that stubs VLC's `vlc` API (widgets +
playback), loads the real
115     extension, drives its callbacks, and asserts the results ? run with **Lua 5.1** (the
interpreter VLC 3.x embeds):
116
117     ```bash
118     lua5.1 test/dialog_test.lua        # exit 0 = pass; covers reason reset, shots column,
numbering, undo, help toggle, Play/Pause toggle
119     ```
120
121     It covers all the logic but not VLC's GUI rendering or real playback (there's no
headless VLC UI automation) ? those
122     are checked by loading in VLC with `vlc -vv --file-logging` (must scan with no `Error
loading` line) plus a manual
123     click-through. See [test/README.md](test/README.md).
124
125     ## Contributing
126     Issues and PRs welcome ? especially live test reports per sport/OS and small UX fixes.
The plugin is a single
127     self-contained Lua file (`vlc/rally_annotator.lua`) using only documented VLC 3.x Lua
APIs; please keep
128     `test/dialog_test.lua` green (`lua5.1 test/dialog_test.lua`) and add a case for new
dialog logic.
```

```
129
130     ## License
131     MIT ? see [LICENSE](LICENSE).
132
```

30. assistant (2026-06-14T08:53:51.774Z)

The `bhi-puser` worktree is on `feat/per-user-promotion-gate` and has an untracked file `backend/eval/per_user_gate.py` ? the construct being built. Let me read it, and the regression promotion gate, and the rally_gate structural guard.

31. user (2026-06-14T08:53:52.725Z)

```
1       """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3       The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4       baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5       held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6       fall back to the shipped baseline (variant A). This module is the decision logic for
that
7       flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8       (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10      The counter-case this guards against (and its mitigations, implemented here):
11
12      1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15      2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18      3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24      Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25      default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26      threaded through). Nothing here changes existing eval behaviour; callers opt in.
27      """
28      from __future__ import annotations
29
30      from dataclasses import dataclass, field
31      from typing import Any, Dict, Sequence
32
33      from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35      __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38      @dataclass
39      class PromotionDecision:
40          """Structured result of a per-user promotion decision.
```

```

        - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
          cue that cleared the full bar).
        - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
          always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
          ``promote``.
        - ``reason`` ? short human-readable explanation of the decision.
        - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
          so a caller can log/audit exactly why a user was or wasn't personalized.
        """

        promote: bool
        keep_on: bool
        reason: str
        n: int
        mean_delta: float
        ci_low: float
        ci_high: float
        ci_excludes_zero: bool
        sign_test: Dict[str, float] = field(default_factory=dict)
        floor_met: bool = False
        structural_protected: bool = False

        def as_dict(self) -> Dict[str, Any]:
            """Plain-dict view (for JSON logging / telemetry)."""
            return {
                "promote": self.promote,
                "keep_on": self.keep_on,
                "reason": self.reason,
                "n": self.n,
                "mean_delta": self.mean_delta,
                "ci_low": self.ci_low,
                "ci_high": self.ci_high,
                "ci_excludes_zero": self.ci_excludes_zero,
                "sign_test": self.sign_test,
                "floor_met": self.floor_met,
                "structural_protected": self.structural_protected,
            }


    def should_promote_for_user(
        a_scores: Sequence[float],
        b_scores: Sequence[float],
        *,
        structural: bool = False,
        min_n: int = 5,
        p_threshold: float = 0.05,
        confidence: float = 0.95,
        n_boot: int = 2000,
        seed: int = 0,
        eps: float = 0.0,
    ) -> PromotionDecision:
        """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.

        ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
for
        variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
        (same order). The decision rules (faithful to the module docstring):

```

```
99          1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
100            ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101            in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
paired
103            ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
videos
104            win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108         """
109         n = min(len(a_scores), len(b_scores))
110         floor_met = n >= min_n
111
112         # --- Evidence (reuse eval_stats; never reinvent the statistics)
----------------------
113         if n == 0:
114             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115             sign_test: Dict[str, float] = paired_sign_test([], eps)
116             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117         else:
118             delta = paired_delta_ci(
119                 a_scores, b_scores,
120                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121             )
122             mean_delta = float(delta["mean_delta"])
123             ci_low = float(delta["ci_low"])
124             ci_high = float(delta["ci_high"])
125             ci_excludes_zero = bool(delta["ci_excludes_zero"])
126             sign_test = delta["sign_test"]
127
128         sign_p = float(sign_test.get("p_value", 1.0))
129         sign_wins = float(sign_test.get("wins", 0.0))
130         sign_losses = float(sign_test.get("losses", 0.0))
131         # The sign test "agrees" with a promotion only if it is significant AND the majority
132         # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133         # never satisfy the promotion bar.
134         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136         # --- Rule 1: structural guards are never disabled on "no lift"
-----------------------
137         if structural:
138             return PromotionDecision(
139                 promote=False,           # nothing to "promote" ? it ships ON by design
140                 keep_on=True,            # ALWAYS kept ON, regardless of measured per-user
lift
141                 reason=(
142                     "structural guard kept ON regardless of measured per-user lift "
143                     "(value is in failure modes this user's corpus may not yet contain)"
144                 ),
145                 n=n,
146                 mean_delta=mean_delta,
147                 ci_low=ci_low,
148                 ci_high=ci_high,
149                 ci_excludes_zero=ci_excludes_zero,
150                 sign_test=sign_test,
151                 floor_met=floor_met,
152                 structural_protected=True,
153             )
154
155         # --- Rule 2: minimum-N floor (refuse before even looking at lift)
```

```
--------------------
156         if not floor_met:
157             return PromotionDecision(
158                 promote=False,
159                 keep_on=False,           # fall back to the shipped baseline (variant A)
160                 reason=(
161                     f"below minimum-N floor (n={n} < min_n={min_n}); "
162                     "falling back to shipped baseline"
163                 ),
164                 n=n,
165                 mean_delta=mean_delta,
166                 ci_low=ci_low,
167                 ci_high=ci_high,
168                 ci_excludes_zero=ci_excludes_zero,
169                 sign_test=sign_test,
170                 floor_met=False,
171                 structural_protected=False,
172             )
173 
174         # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
----------------
175         if ci_excludes_zero and sign_agrees:
176             promote = True
177             reason = (
178                 f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179                 f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180                 f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181             )
182         else:
183             promote = False
184             misses = []
185             if not ci_excludes_zero:
186                 misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187             if not sign_agrees:
188                 misses.append(
189                     f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}, "
190                     f"p={sign_p:.4f})"
191                 )
192             reason = (
193                 "insufficient evidence (" + "; ".join(misses) + "); "
194                 "falling back to shipped baseline"
195             )
196 
197         return PromotionDecision(
198             promote=promote,
199             keep_on=promote,             # non-structural: keep_on tracks promote exactly
200             reason=reason,
201             n=n,
202             mean_delta=mean_delta,
203             ci_low=ci_low,
204             ci_high=ci_high,
205             ci_excludes_zero=ci_excludes_zero,
206             sign_test=sign_test,
207             floor_met=True,
208             structural_protected=False,
209         )
210 
```

32. user (2026-06-14T08:53:53.005Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2 
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
```

```
    wrapper over machinery that already exists:

        ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
        predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
        comparison    ? interval matching + P/R/F1                    [backend.eval.metrics]
        baseline      ? a per-video snapshot of "known good" numbers [this module]

    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
    """
    import os
    import json
    import hashlib
    import logging
    from dataclasses import dataclass, asdict
    from typing import Dict, List, Optional

    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

    logger = logging.getLogger(__name__)

    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
    # actually load a committed baseline to compare against.
    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
        """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store
```

```
    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                      gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
        as incommensurable rather than silently compared.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
            "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)
        fp = gt_hash(ground_truth)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        has_baseline = b is not None
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
```

```
            # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
            # against different labels is not a valid reference ? flag it instead of
trusting it.
            b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
            if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
                regressed = True
                reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
            if b_det is not None and b_det != detector:
                regressed = True
                reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
            if b_hash is not None and b_hash != fp:
                regressed = True
                reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
            has_baseline=has_baseline, gt_hash=fp,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)


    def format_result(r: RegressionResult) -> str:
        verdict = "REGRESSED" if r.regressed else "ok"
        head = (f"[{verdict}] {r.video_id} ({r.stage}) "
                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
```

```
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189
```

33. assistant (2026-06-14T08:53:57.368Z)

I now have a thorough understanding. Let me check a couple more things: how the segmenter base/registry reads config (the served config path) and whether there are existing tests for the fusion segmenter, plus the trajectory_hybrid base to understand `_select_for_handoff` flow. Let me also check the bhi-puser scratch for any prior per-user design docs.

34. user (2026-06-14T08:53:59.874Z)

```
tests\__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
tests\test_annotation_providers.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
tests\test_validators_base.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_tracknet_runner.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
tests\test_reingest_safety.py
tests\test_eval_metrics.py
tests\test_compiler.py
tests\test_eval_annotations.py
tests\test_gt_loader.py
tests\test_import_smoke.py
tests\test_config_wasb.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
tests\test_base.py
tests\test_eval_classifier.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_path_safety.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_training_gen0.py
tests\test_video.py
tests\test_wasb_cache.py
tests\test_tracknet_hybrid.py
tests\test_wsl_tilde_quoting.py
tests\test_upload_download.py
tests\test_api_endpoints.py
tests\test_watermark.py
tests\test_cleanup_caches.py
```

```
tests\test_wasb_runner.py
tests\test_experiment.py
tests\test_yolo_hybrid.py
tests\test_fusion_features.py
tests\test_fusion_hybrid.py
tests\test_rally_gate.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_config.py
tests\test_logging_setup.py
tests\test_rally_summary.py
tests\test_audio_features.py
tests\test_eval_stats.py
tests\test_person_detector.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_fusion_compare.py
tests\test_async_jobs.py
tests\test_database.py
tests\test_execution_policy.py
tests\test_experiment_honest.py
tests\test_job_self_scheduling.py
tests\test_run_telemetry.py
tests\test_variant_tuning.py
tests\test_fusion_golden_telemetry.py
```

35. user (2026-06-14T08:53:59.958Z)

```
analyze_motion.py
clean_state.py
diagnose_rally.py
gemini_video_debugger.py
list_models.py
test_ai_segmenter.py
test_thresholds.py
===tests for fusion/experiment/per_user===
test_distill_local.py
test_eval_stats.py
test_experiment.py
test_experiment_honest.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
---glob tests---
```

36. assistant (2026-06-14T08:54:04.146Z)

I have everything I need. Let me read the trajectory_hybrid base to understand exactly where the served decision path reads config (the bridge point for per-user model serving), and the existing test_fusion_hybrid to model the test approach.

37. user (2026-06-14T08:54:04.827Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
```

```
                        the ``<family>-hybrid-<model>`` detector tag in metadata.
    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
    - ``name`` / ``description`` properties (the registry contract).
    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                        detection_params extras such as weights_path).

    This base is deliberately NOT registered; only concrete subclasses register.
    """
    import os
    import time
    import logging
    import concurrent.futures
    from typing import Any, Dict, List, Optional, Tuple

    import cv2

    from backend.pipeline.segmenters.base import BasePlaySegmenter
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
```

```
75                 return max(3, int(duration / 0.8))
76             try:
77                 return int(shot_count)
78             except (ValueError, TypeError):
79                 return max(3, int(duration / 0.8))
80
81         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
82         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                                     frame_width: float, video_path: str,
84                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
86             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88             features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89             return {
90                 "peak_velocity": cw["peak_velocity"],
91                 "mean_velocity": cw["mean_velocity"],
92                 "active_frame_count": cw["active_frame_count"],
93                 "track_id_count": cw["track_id_count"],
94             }
95
96         def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                                 window_stats: List[Dict[str, Any]],
98                                 idx_cfg: Dict[str, Any]) -> List[int]:
99             """Indices of the candidate windows that proceed to the AI handoff (and thus may
100            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102            Persisted candidates are NOT affected ? they remain the full superset for
offline
103            re-scoring."""
104            return list(range(len(windows)))
105
106        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
107                          player_pool: Optional[List[str]] = None,
108                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109            # override-ignore: the ABC declares the Result contract (Success|Failure)
110            # but every live segmenter still returns a bare list ? the documented
111            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112            label = self.display_label
113            # --- Sport profile + AI provider wiring (identical across segmenters) ---
114            sport_name = self.config.get("sport", "badminton")
115            try:
116                sport_profile = sport_registry.get(sport_name)
117            except KeyError as e:
118                logger.error(str(e))
119                return []
120
121            idx_cfg = self.config.get("indexing", {})
122            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125            api_key_env_var = f"{provider_name.upper()}_API_KEY"
126            api_key = os.environ.get(api_key_env_var)
127            if not api_key:
128                logger.error(
129                    f"{api_key_env_var} environment variable is not set! "
130                    f"To run the {provider_name} handoff, set your API key. "
131                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132                )
```

```
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            runner, runner_params = self._build_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
```

```
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
```

```
250                 try:
251                     os.remove(w_path)
252                 except OSError:
253                     pass
254                 return valid
255 
256             ai_segments: List[dict] = []
257             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258                 futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                            for wi in handoff_indices]
260                 for future in futures:
261                     ai_segments.extend(future.result())
262             try:
263                 os.rmdir(handoff_dir)
264             except OSError:
265                 pass
266             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267 
268             # --- Phase 4: DB population + candidate linking ---
269             segments_data = []
270             for segment in ai_segments:
271                 try:
272                     start = float(segment["start_time"])
273                     end = float(segment["end_time"])
274                     dur = end - start
275                     metadata = {
276                         "shot_count": self._shot_count_from(segment, dur),
277                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                         "detector": f"{self.family}-hybrid-{model_name}",
279                     }
280                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                     if segment_id:
282                         candidate_id = segment.get("_candidate_id")
283                         if candidate_id is not None:
284                             self.db.link_candidate_to_segment(candidate_id, segment_id)
285                         segments_data.append({
286                             "id": segment_id, "start_time": start, "end_time": end,
287                             "duration": dur, "metadata": metadata,
288                         })
289                 except Exception as e:
290                     logger.warning("Skipping invalid segment struct: %s", e)
291 
292             return segments_data
293 
```

38. assistant (2026-06-14T08:54:10.088Z)

I now have a complete picture of the architecture. The key serving-bridge insight: `_select_for_handoff` in `fusion_hybrid.py` is the exact production decision seam ? it reads `idx_cfg["fusion"]` thresholds/cue_flags but never loads a `LogisticRegression`. That's where the per-user model bridge must be wired.

Let me verify one detail ? how config is loaded/merged (to design the per-user config overlay) ? and check the existing fusion test to model the test approach.

39. user (2026-06-14T08:54:10.644Z)

backend\config\loader.py
backend\config__init__.py
backend\config\models.py

40. user (2026-06-14T08:54:11.429Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37          return r
38
39
40      def _run(indexing=None, provider_segments=None):
41          provider = MagicMock()
42          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43          db = MagicMock()
44          db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45          db.add_play_segment.return_value = 200
46
47          with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48               patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
49               patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50               patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51               patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
```

```
52               patch("os.environ.get", return_value="dummy_key"):
53              preg.get.return_value = provider
54              seg = FusionSegmenter(db, {"indexing": indexing or {}})
55              res = seg.process_video("v1", "clip.mp4")
56              return db, provider, res
57
58
59      def _stats_calls(db):
60          """Persisted stats dicts, in call order."""
61          return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64      def test_net_crossings_persisted_and_person_off_by_default():
65          db, _provider, _res = _run()
66          stats = _stats_calls(db)
67          assert len(stats) == 2                              # both windows persisted
(superset)
68          assert all("net_crossings" in s for s in stats)
69          # Window A (rally) crosses many times; window B (held) zero.
70          assert stats[0]["net_crossings"] >= 2
```

41. assistant (2026-06-14T08:54:14.261Z)

I have a complete and grounded understanding of the codebase. Let me read the config loader briefly to finalize the per-user config overlay design, then deliver the plan.

42. user (2026-06-14T08:54:15.122Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
32          """
33          unknown: list[str] = []
34          fields = model_cls.model_fields
35          for key, val in data.items():
36              if key not in fields:
```

```
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by setup.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
```

```
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatible alias used by some older call sites during transition.
105     def get_default_config() -> dict[str, Any]:
106         """Return defaults as a plain dict (for transitional use only)."""
107         return _get_builtin_defaults()
108
```

43. assistant (2026-06-14T08:54:18.053Z)

I have everything needed. The architecture is fully grounded. Let me mark this chapter and deliver the complete plan.

44. user (2026-06-14T08:54:19.290Z)

Chapter marked: "Personalization feature plan". Continue your current work.

45. assistant (2026-06-14T08:55:59.694Z)

I have fully explored the codebase. One note before the plan: I was asked to write the design to `scratch/PERSONALIZATION_FEATURE_DESIGN.md`, but I'm in read-only planning mode and cannot create files. The complete design follows here as my response ? the parent agent / owner can persist it.

A key real-code finding worth flagging up front: the per-user promotion gate is **already written** as an untracked file in the `bhi-puser` worktree at `C:\Users\avidu\Projects\bhi-puser\backend\eval\per_user_gate.py` (branch `feat/per-user-promotion-gate`, which currently has zero committed delta vs `master`). It is pure, tested-shaped, and reuses `eval_stats`. So construct #3 is "commit + integrate," not "build from scratch."

Executive Summary (~300 words)

The owner's HYBRID ? generalization-first baseline + flag-gated per-user personalization + a contribution flywheel ? is buildable on the existing harness because the codebase already cleanly separates **expensive detection** (persisted once into `candidate_segments.stats`) from **cheap, swappable decision** (offline re-scoring via `experiment.Variant`). Personalization is per-user re-scoring of already-persisted features. The four real walls, all verified in code, are: (1) `LogisticRegression.load` is **never called** in `backend/pipeline/` ? the served decision seam (`fusion_hybrid._select_for_handoff`) reads only Gate thresholds, so a learned per-user model has no serving path; (2) `human_label` is a **read-only stub** (column + `only_unlabeled` filter, no setter, no API) ? corrections can't become labels; (3) there is **no user/tenant dimension** anywhere; (4) the validated day-0 win (label-free Gate-A/B fusion, ?F1 +0.074, 6/6, p=0.031) is opt-in but not shipped as the default battery.

The plan sequences these honestly. **Phase 0** (this weekend, mostly engineering) commits the already-written `per_user_gate.py`, ships Gate-A as the validated day-0 battery, and adds a held-out-USER eval scaffold + per-user no-regression guard ? pure-Python, no schema, no API risk. **Phase 1** adds the per-user store (migration v4), a per-user config overlay, and the serving bridge that wires `LogisticRegression.load` + the promotion gate into `_select_for_handoff`. **Phase 2** builds the correction-loop write path (confirm/reject API ? `human_label`), the recall-protecting de-bias prior, and the consent-ledgered contribution flywheel. **Phase 3** integrates the rally-annotator as a user-as-annotator active-learning loop, shot taxonomy, and baseline-upgrade governance.

Every per-user promotion rides the existing honest small-N bar (`paired_delta_ci` + sign test) with a hard min-N floor, falling back to the baseline below it. The headline "<5 videos" becomes a falsifiable held-out-USER release criterion, not a slogan.

Architecture

The two-layer decoupling (already real)

- **Detection layer** (GPU, run once): `FusionSegmenter` persists per-window features into `candidate_segments.stats` ? `net_crossings` always, person-cue features when `cue_flags.person` is on. The persisted set is always the **full superset**; only the served handoff is gated.
- **Decision layer** (CPU, swappable): `experiment.Variant` + `experiment.predict()` re-score persisted features into intervals. Personalization == a **per-user Variant** (cue_flags + min_crossings/min_players + optional LogReg + threshold).

The single highest-leverage wiring

`fusion_hybrid.py::_select_for_handoff` (lines 133?151) is the **exact production decision seam**. Today it reads `idx_cfg["fusion"]` thresholds/cue_flags. The serving bridge resolves a **per-user FusionConfig overlay** here and, when a per-user model is pinned and the promotion gate says ON, applies `LogisticRegression.predict_proba` over `window_stats` ? the same math `experiment.predict()` already runs offline. This is the one place that closes gap #1.

Per-user config overlay (no fork of the pipeline)

A per-user model is a resolved `FusionConfig` + an optional LogReg JSON path + a version. The bridge produces a per-request `idx_cfg["fusion"]` by overlaying the user's store row on the served defaults. `_select_for_handoff` and `_enrich_candidate_stats` already consume `idx_cfg["fusion"]` ? no segmenter rewrite, only an overlay resolver upstream.

Per-user data model + schema migrations

All migrations follow the established pattern in `database.py` (`PRAGMA user_version`, ordered `if version < N` blocks; tests assert the version). Current version is **3**; personalization starts at **v4**.

```
-- v4: tenant dimension (nullable user_id everywhere ? single-user stays NULL, zero behavior change)
ALTER TABLE videos            ADD COLUMN user_id TEXT;          -- NULL = legacy/local single-user
ALTER TABLE candidate_segments ADD COLUMN user_id TEXT;         -- denormalized for per-user label queries
CREATE INDEX idx_videos_user            ON videos(user_id);
CREATE INDEX idx_candidates_user_label  ON candidate_segments(user_id, human_label);

-- v5: per-user model store (versioned, survives baseline upgrades)
CREATE TABLE user_models (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT NOT NULL,
    version INTEGER NOT NULL,             -- per-user monotonic
    baseline_version TEXT NOT NULL,       -- which shipped battery this was fit against
    feature_schema_hash TEXT NOT NULL,    -- MAJOR-upgrade invalidation key (5-feature LogReg vs ActionFormer)
    fusion_config TEXT NOT NULL,          -- resolved FusionConfig JSON (cue_flags + min_crossings/min_players)
    classifier_json TEXT,                 -- LogisticRegression.to_dict() inline, NULL = tuning-only (no learned head)
    promotion_evidence TEXT,              -- PromotionDecision.as_dict() per cue (the trust-feature payload)
    n_videos_at_fit INTEGER NOT NULL,     -- corpus size ? gates min-N floor + cold-start fallback
    is_active INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES ... -- (users table arrives with real auth; user_id is a free TEXT key until then)
);
```

```
CREATE UNIQUE INDEX idx_user_models_active ON user_models(user_id) WHERE is_active = 1;

-- v6: consent ledger (flywheel governance ? enforced, not advisory)
CREATE TABLE contribution_consent (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT NOT NULL,
    scope TEXT NOT NULL,                 -- 'data' | 'video' | 'diagnostics' | 'annotation'
    granted INTEGER NOT NULL,            -- explicit opt-in; default-deny
    is_minor_attested INTEGER NOT NULL DEFAULT 0,  -- minors-exclusion gate (pooling refused if
1)
    granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    revoked_at TIMESTAMP
);
CREATE INDEX idx_consent_user_scope ON contribution_consent(user_id, scope);
```

Design notes:
- **`user_id` nullable everywhere** ? the single-user local install is `user_id IS NULL`, byte-identical to today. This is the "no tenant dimension" gap closed without breaking the local-first promise.
- **`feature_schema_hash`** is the MINOR-vs-MAJOR upgrade switch: a MINOR battery upgrade carries the user's `fusion_config` + `classifier_json` byte-identically; a MAJOR (the planned M0.4 ActionFormer swap) changes the hash ? every per-user model is flagged dimensionally invalid and must re-fit from stored correction labels.
- **`classifier_json` inline** (not a file path) keeps the store self-contained and tenant-safe; `LogisticRegression.from_dict` already round-trips it.

The serving bridge (closing gap #1)

New module `backend/pipeline/personalization/serving.py`:
- `resolve_user_fusion_config(db, user_id, served_cfg) -> dict` ? load the active `user_models` row, overlay its `fusion_config` onto `served_cfg["fusion"]`; if no active model or `n_videos_at_fit < min_n`, return the **shipped baseline** (cold-start fallback = Gate-A/B day-0 battery).
- `load_user_classifier(db, user_id) -> Optional[LogisticRegression]` ? `LogisticRegression.from_dict(json.loads(row["classifier_json"]))`. **This is the first production caller of the load path.**

Wiring point in `fusion_hybrid.py::_select_for_handoff`: after the existing Gate-A/B masking, if a per-user classifier is resolved AND its promotion evidence says ON, score `window_stats` through `model.predict_proba` and drop sub-threshold windows ? reusing the exact feature-matrix logic from `experiment.predict()`. Persisted candidates stay the full superset (unchanged invariant). Default path (no `user_id`, no model) is untouched.

APIs (`backend/main.py`)

- `POST /api/candidates/{candidate_id}/label` ? sets `human_label` ('confirm'|'reject') ? **the first production writer of `human_label`**, closing gap #2. Body carries `user_id`.
- `POST /api/users/{user_id}/retune` ? 202 job: re-run per-user LOVO over the grown corpus, fit/refresh the `user_models` row, run the per-user no-regression guard, activate only if it passes.
- `GET /api/users/{user_id}/model` ? the trust-feature payload: per-cue ON/OFF + promotion evidence ("we tested motion on YOUR videos ? it didn't help, so it's off").
- `POST /api/users/{user_id}/consent` ? write `contribution_consent` (scope + minor attestation).
- `GET /api/users/{user_id}/annotation-task` ? active-learning: a ~2-min high-uncertainty segment from the user's own footage for rally-annotator labeling.

All reuse the existing async job model (`create_job`/`claim_due_job`) for the slow retune path.

Each construct, mapped to real code

1. **Per-user store + serving bridge** ? migration v4/v5 + `serving.py` + the `_select_for_handoff` wiring above. Per-user versioning survives baseline upgrades via `baseline_version` + `feature_schema_hash`.
2. **Correction-loop ? per-user labels** ? the `POST .../label` setter + a new `db.set_human_label(candidate_id, user_id, label)`. The **de-bias prior**: corrections are false-positive-biased (users reject FPs, rarely flag silent misses), so the per-user fit applies a recall-protecting class prior (the existing `class_weight="balanced"` in `classifier.py` is the hook; add a configurable miss-inflation factor) and the active-learning loop (construct 5) surfaces high-uncertainty *unfired* windows to recover misses.
3. **Per-user promotion gate** ? **commit the already-written** `per_user_gate.py::should_promote_for_user` (min-N floor + `structural=True` exemption for Gate-A). Surface `PromotionDecision.as_dict()` as the trust feature via `GET .../model`.
4. **Contribution flywheel** ? consent ledger (v6) + opt-in upload reusing the existing chunked-upload endpoints, scoped per-user. Payoff-scales: a multi-video user with reproduced capture settings gets a fit `user_models` row; a single-video user falls below `min_n` and **leans on the baseline** (the gate enforces this automatically). Minors-exclusion blocks pooling. Baseline is owner-trained on GPU; flywheel feeds it only via consented pooling + an optional federated coefficient nudge (deferred, counsel-gated).
5. **User-as-annotator charter** ? integrate `rally-annotator` (VLC Lua plugin, CSV `rally_number,start_time,end_time,ending_reason,sport,shots_count`). Trigger: when a user crosses a corpus threshold or the gate is starved below `min_n`. Pick a high-uncertainty ~2-min region (LogReg proba near 0.5 over persisted windows), guide labeling, ingest the CSV as per-user golden labels feeding retune (and, with consent, the baseline).
6. **Per-user eval** ? `backend/eval/per_user_eval.py`: held-out-USER learning curve (sweep k=1..K of a user's videos, compare battery v_old vs v_new with `paired_delta_ci` + sign test) as the falsifiable "<5 videos to superior quality" release criterion; a per-user no-regression guard analogous to `regression.py` but against a frozen per-user CONTROL.
7. **Day-0 batteries** ? flip the validated label-free Gate-A fusion to the shipped default battery (config + a promotion record), zero per-user labels. Baseline-upgrade governance: MINOR (carry per-user model byte-identical) vs MAJOR (re-fit) keyed on `feature_schema_hash`.

Governance

- **Consent is default-deny and enforced** at the pooling query (`WHERE granted=1 AND is_minor_attested=0`), not advisory.
- **Privacy sequencing honored**: do not advertise "data never leaves your machine" until on-device ships; near-term pooling carries the full cloud posture. Per-user delete = delete `user_models` + `candidate_segments` rows for `user_id` (CASCADE already wired on videos).
- **Roora**: re-scoped to certify each battery upgrade against an independent vendor-graded golden set + seed cold-start; spend does not shrink.

PHASED PR-SIZED BUILD PLAN

Phase 0 ? Safety + day-0 batteries (THIS WEEKEND, rehearsable, mostly engineering)

No schema, no API, no served-pipeline behavior change. All pure-Python + tests.

- **PR-0.1 ? Commit the per-user promotion gate.** `git add` the already-written `bhi-puser/backend/eval/per_user_gate.py` onto `feat/per-user-promotion-gate`. *Files:* `backend/eval/per_user_gate.py`. *Tests:* `tests/test_per_user_gate.py` (min-N floor refuses at n<5; structural guard keeps-ON regardless of delta; CI-excludes-0 + sign-test agreement promotes; significant net-loss never promotes). **Engineering.**
- **PR-0.2 ? Held-out-USER eval scaffold.** New `backend/eval/per_user_eval.py`: learning-curve sweep over a user's video list reusing `experiment.compare` + `eval_stats.paired_delta_ci`. *Tests:* `tests/test_per_user_eval.py` (synthetic corpus; asserts the smallest-k crossing

logic). **Engineering.**
- **PR-0.3 ? Per-user no-regression guard.** Extend `regression.py` (or a thin sibling) with a per-user CONTROL baseline keyed by `user_id::stage`. *Tests:* `tests/test_per_user_regression.py`. **Engineering.**
- **PR-0.4 ? Ship Gate-A as the day-0 battery.** Set `fusion.min_crossings` default + record the validated promotion (?F1 +0.074, 6/6, p=0.031) to the leaderboard; document MINOR/MAJOR upgrade rule. *Files:* `backend/config/models.py` (FusionConfig default), `eval_baselines/experiment_leaderboard.json`, a docs note. *Tests:* `tests/test_config.py` + `tests/test_fusion_hybrid.py` (Gate-A now drops the held-shuttle window by default). **Owner-gated** (changes shipped default behavior ? needs owner sign-off + a regression run on the n=6 golden set).
- **PR-0.5 ? Within-user LOVO measurement harness.** The analysis's "fund this first" experiment: a runnable script re-scoring within-user splits to confirm/kill the core statistical premise. *Files:* `scripts/within_user_lovo.py`. *Tests:* logic-level only. **Owner-gated** (runs on the GPU/WSL box).

Phase 1 ? Per-user tuning serving + store

- **PR-1.1** ? Migration v4 (nullable `user_id`) + `db.set_human_label`/`set_user_id` plumbing. *Tests:* `tests/test_database.py` (version asserts, NULL-user back-compat).
- **PR-1.2** ? Migration v5 `user_models` + CRUD (`upsert_user_model`, `get_active_user_model`). *Tests:* `tests/test_user_models.py`.
- **PR-1.3** ? `serving.py` overlay resolver + `load_user_classifier`. *Tests:* `tests/test_serving_bridge.py`.
- **PR-1.4** ? Wire the bridge into `_select_for_handoff` (first production `LogisticRegression` use). *Tests:* extend `test_fusion_hybrid.py` (per-user model ON drops a window; default path unchanged). **Owner-gated** (touches served path).
- **PR-1.5** ? `POST /api/users/{user_id}/retune` + `GET .../model` trust feature. *Tests:* `tests/test_api_endpoints.py`.

Phase 2 ? Correction loop + flywheel

- **PR-2.1** ? `POST /api/candidates/{id}/label` setter (closes gap #2). *Tests:* `test_api_endpoints.py`.
- **PR-2.2** ? De-bias prior in the per-user fit (recall-protecting miss-inflation). *Tests:* `test_eval_classifier.py`.
- **PR-2.3** ? Migration v6 consent ledger + `POST .../consent` + minors-exclusion enforcement. *Tests:* `tests/test_consent_ledger.py`.
- **PR-2.4** ? Opt-in contribution upload (reuse chunked upload, per-user scoping) + payoff-scaling fallback. **Owner-gated** (privacy surface).

Phase 3 ? Annotator charter + taxonomy + baseline upgrades

- **PR-3.1** ? Active-learning task picker (`GET .../annotation-task`, high-uncertainty ~2-min segment). *Tests:* `tests/test_annotation_task.py`.
- **PR-3.2** ? rally-annotator CSV ingest ? per-user golden labels. *Tests:* `tests/test_annotation_ingest.py`.
- **PR-3.3** ? Coarse hard/soft shot proxy from existing speed/arc/reversal signals, default-OFF, measured on golden. **Owner-gated.**
- **PR-3.4** ? Baseline-upgrade governance (MINOR carry / MAJOR re-fit on `feature_schema_hash`) + optional federated nudge (deferred, counsel-gated). **Owner-gated.**

Honest risks (from the analysis, carried forward)

- **Small-N eval noise** is central and *worse* per-user (n=1?15). Mitigation is structural: the gate refuses below `min_n` and falls back to Gate-A/B; never auto-promotes in the regime the tooling exists to refuse in. PR-0.5 confirms-or-kills the premise before any serving build-out.
- **Cold-start** kills the median user; the day-0 Gate-A battery (PR-0.4) is the zero-label floor.
- **Privacy deferral** ? near-term pooling carries full cloud posture; do not advertise on-device until it ships.
- **Compute** ? VLM personalization stays out of scope; v1 is the cheap LogReg/threshold/cue-ON-OFF layer only.

Top 5 open design questions for the owner

1. **`user_id` identity source before real auth.** Phase 1 uses a free-text `user_id` (NULL = local). Is a lightweight device/install UUID acceptable for the alpha, or must real auth land first (which reorders Phase 1 behind an auth PR)?
2. **Where does the per-user model physically live?** Inline `classifier_json` in `user_models` (tenant-safe, self-contained, my recommendation) vs a per-user file under `output/` (matches `distill_local`'s save/load but reintroduces path scoping). This decides the serving-bridge load path.
3. **Min-N floor value and structural-guard list.** `per_user_gate.py` defaults `min_n=5`. Is 5 the ship value, and is Gate-A the *only* `structural=True` guard, or does Gate-B (person) also ship structurally-protected?
4. **Day-0 default flip (PR-0.4) authorization.** Making Gate-A the shipped default changes served behavior for every user. Approve flipping `fusion.min_crossings` default now, or keep it opt-in until a fresh golden-set regression run is re-confirmed on the current corpus?
5. **Flywheel default for the single-user local install.** Should the local/`NULL`-user install default to **fully local, zero pooling** (strongest privacy story, weakest baseline flywheel) until consent is explicitly granted ? and does the "payoff scales with contribution" framing get surfaced in-product, or stay a backend property?

Critical Files for Implementation

- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py (the `_select_for_handoff` serving seam where `LogisticRegression.load` must be wired)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py (schema migrations v4?v6, `human_label` setter, per-user model store)
- C:\Users\avidu\Projects\bhi-puser\backend\eval\per_user_gate.py (the already-written per-user promotion gate to commit + integrate)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py (`Variant`/`predict`/`compare` ? the per-user re-scoring + serving-config contract)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py (the API surface for confirm/reject labels, retune, consent, annotation tasks)